计算机视觉

第05章 机器学习与图像分类

软件学院：刘春

Ref: EECS 498-007, Deep Learning for Computer Vision
Prof. Justin Johnson

本章内容

- 5.1.为什么需要学习
- 5.2.最近邻方法
- 5.3.线性分类器
- 5.4.损失函数与正则化
- 5.5.最优化

5.1.为什么需要学习

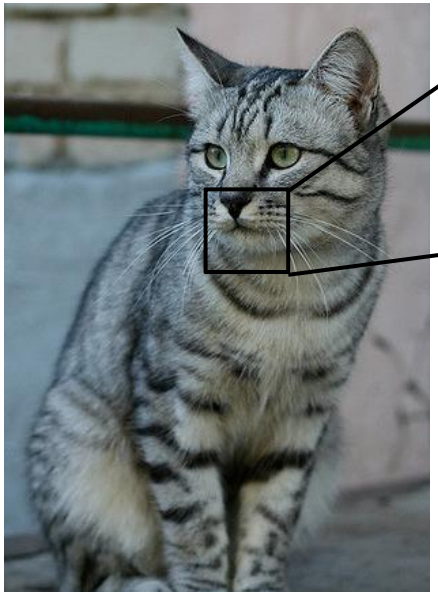
- **从图像分类问题谈起**

输入： 图像

输出： 图像的类别

猫
鸟
鹿
狗
卡车

问题: 语义鸿沟



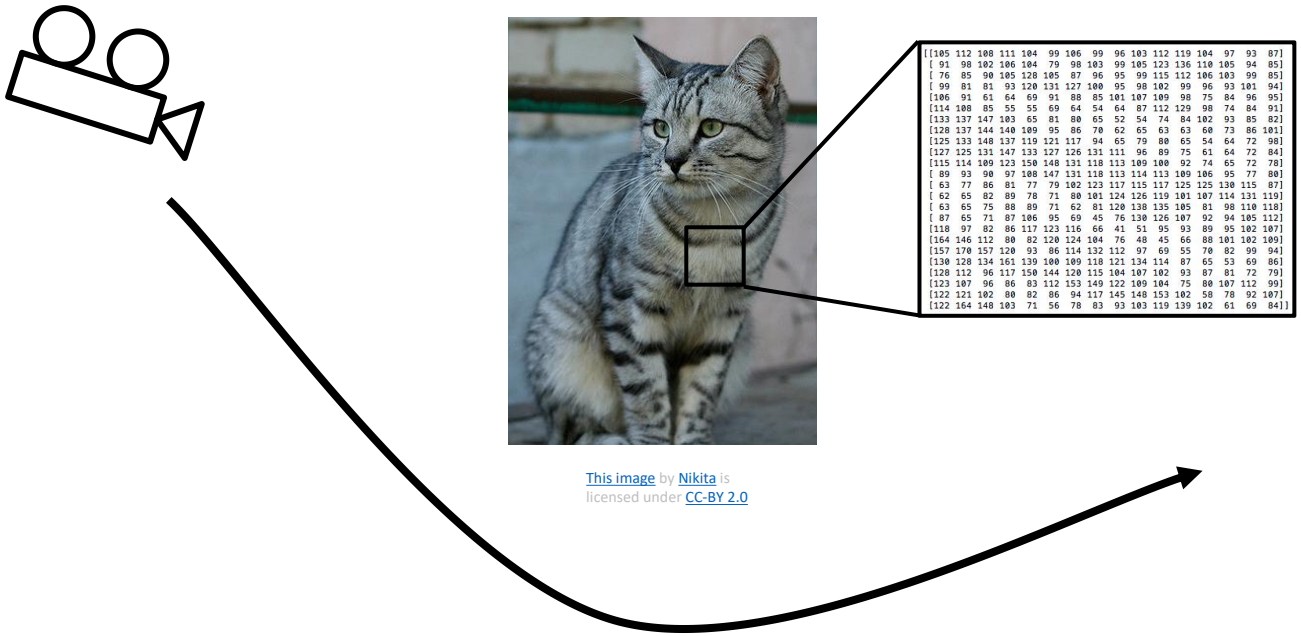

```
[[105 112 108 111 104  99 106  99  96 103 112 119 104  97  93  87]
 [ 91  98 102 106 104  79  98 103  99 105 123 136 110 105  94  85]
 [ 76  85  90 105 128 105  87  96  95  99 115 112 106 103  99  85]
 [ 99  81  81  93 120 131 127 100  95  98 102  99  96  93 101  94]
 [106  91  61  64  69  91  88  85 101 107 109  98  75  84  96  95]
 [114 108  85  55  55  69  64  54  64  87 112 129  98  74  84  91]
 [133 137 147 103  65  81  80  65  52  54  74  84 102  93  85  82]
 [128 137 144 140 109  95  86  70  62  65  63  63  60  73  86 101]
 [125 133 148 137 119 121 117  94  65  79  80  65  54  64  72  98]
 [127 125 131 147 133 127 126 131 111  96  89  75  61  64  72  84]
 [115 114 109 123 150 148 131 118 113 109 100  92  74  65  72  78]
 [ 89  93  90  97 108 147 131 118 113 114 113 109 106  95  77  80]
 [ 63  77  86  81  77  79 102 123 117 115 117 125 125 130 115  87]
 [ 62  65  82  89  78  71  80 101 124 126 119 101 107 114 131 119]
 [ 63  65  75  88  89  71  62  81 120 138 135 105  81  98 110 118]
 [ 87  65  71  87 106  95  69  45  76 130 126 107  92  94 105 112]
 [118  97  82  86 117 123 116  66  41  51  95  93  89  95 102 107]
 [164 146 112  80  82 120 124 104  76  48  45  66  88 101 102 109]
 [157 170 157 120  93  86 114 132 112  97  69  55  70  82  99  94]
 [130 128 134 161 139 100 109 118 121 134 114  87  65  53  69  86]
 [128 112  96 117 150 144 120 115 104 107 102  93  87  81  72  79]
 [123 107  96  86  83 112 153 149 122 109 104  75  80 107 112  99]
 [122 121 102  80  82  86  94 117 145 148 153 102  58  78  92 107]
 [122 164 148 103  71  56  78  83  93 103 119 139 102  61  69  84]]
```

计算机输入

一副图像仅仅是一个数值矩阵:

e.g. 800 x 600 x 3
(3 通道 RGB图像)

挑战：视角变化



所有的像素值随相机移动变化!

挑战： 类内变化

挑战：精细分类

Maine Coon

This image is free for for use under the Pixabay License

Ragdoll

This image is CC0 public domain

American Shorthair

This image is CC0 public domain

挑战：背景杂波

This image is CC0 1.0 public domain

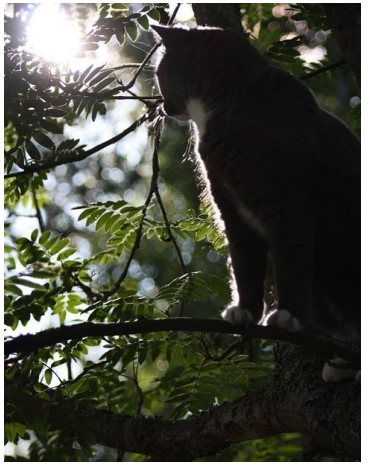
This image is CC0 1.0 public domain

挑战：光照变化

This image is CC0 1.0 public domain

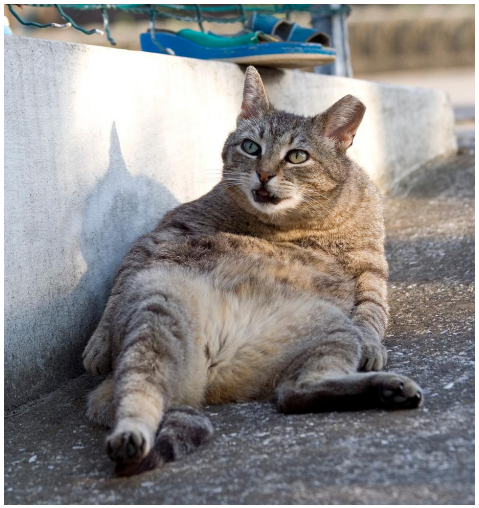
This image is CC0 1.0 public domain

This image is CC0 1.0 public domain

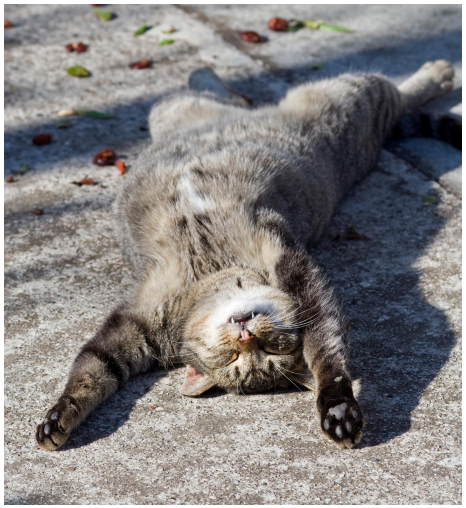


This image is CC0 1.0 public domain

挑战：形变

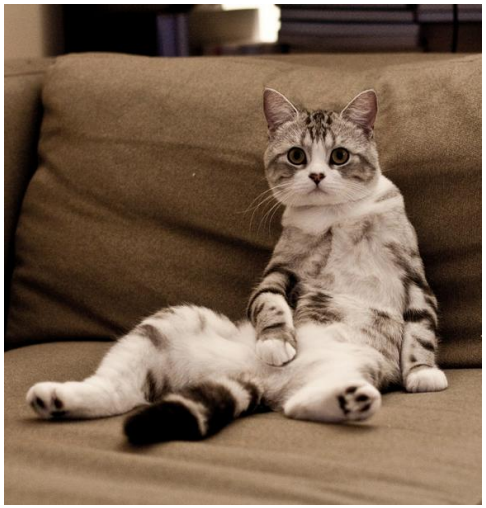


This image by Umberto Salvagnin is licensed under CC-BY 2.0



This image by Umberto Salvagnin is licensed under CC-BY 2.0

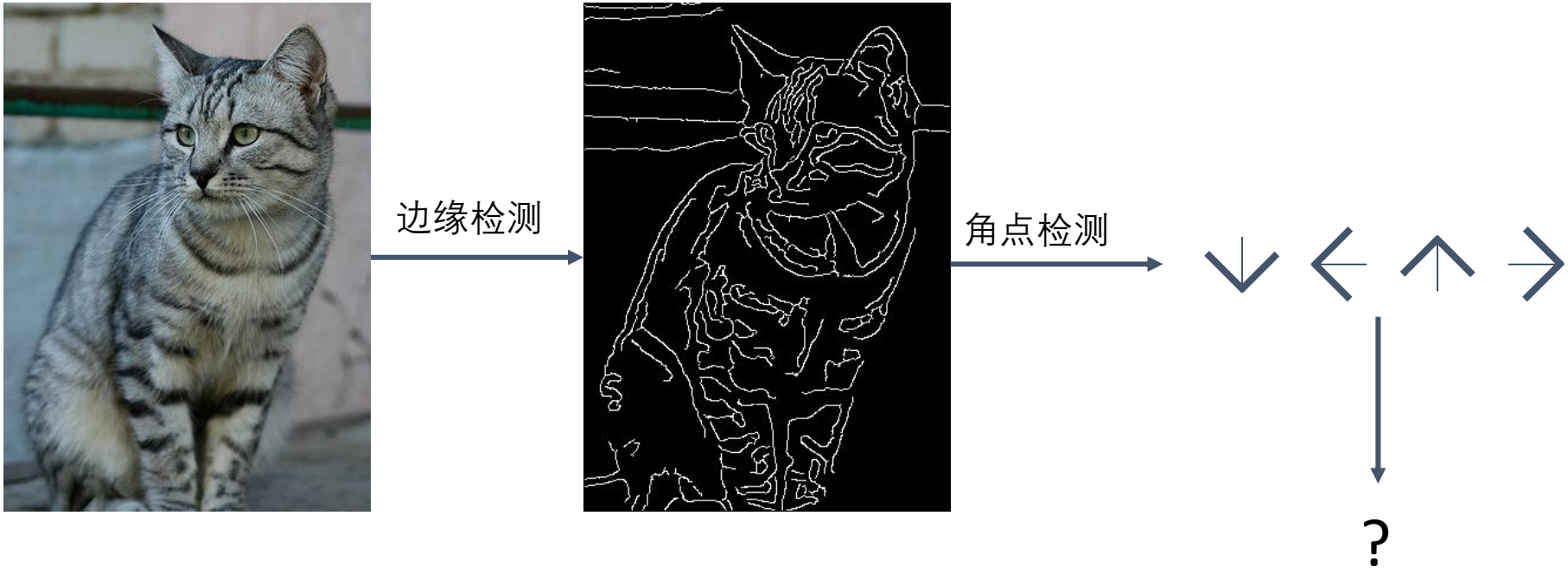
This image by sare bear is licensed under CC-BY 2.0



This image by Tom Thai is licensed under CC-BY 2.0

挑战：遮挡

基于特征提取的传统方法…



John Canny, “A Computational Approach to Edge Detection”, IEEE TPAMI 1986

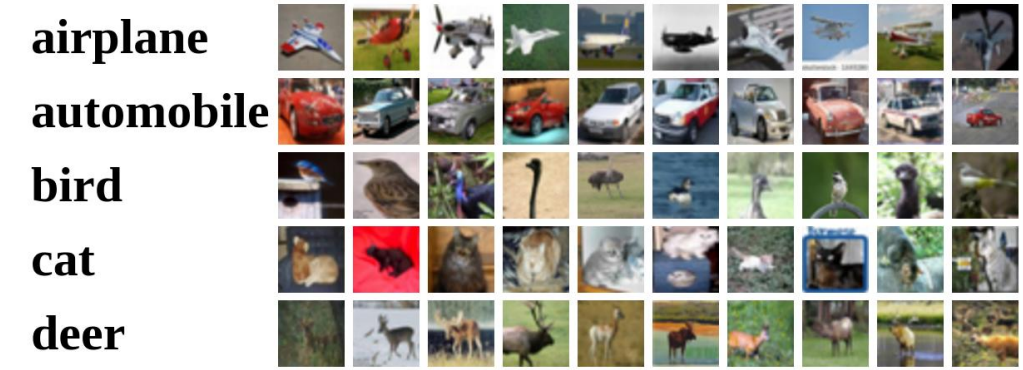
机器学习：数据驱动方法

1. 收集图像和对应标签的数据集
2. 使用机器学习去训练分类器
3. 使用新图像验证分类器

```
def train(images, labels):
  # Machine learning!
  return model
```

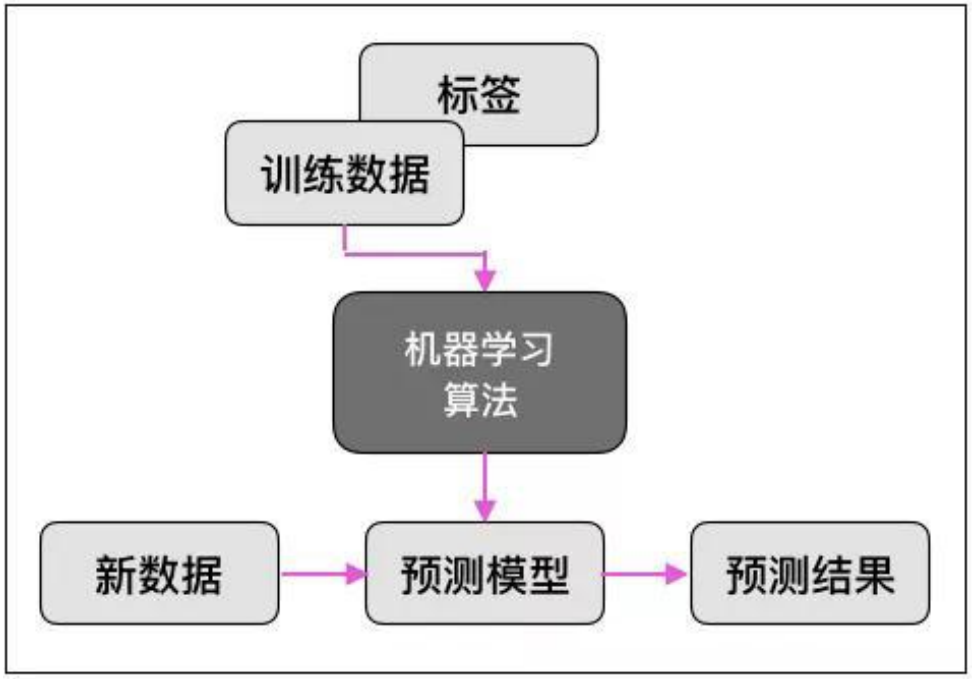
```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

举例： 训练集



监督学习

- 目的：对于输入数据提供一个最佳预测输出



本章内容

- 5.1.为什么需要学习
- 5.2.最近邻方法
- 5.3.线性分类器
- 5.4.损失函数与正则化
- 5.5.最优化

最基本的分类器：最近邻分类

```
def train(images, labels):
  # Machine learning!
  return model
```

存储所有图像和对应的标签

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

预测为最相似的训练图像的标签

图像比较**距离测度**

L1 距离:

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

test image					training image					pixel-wise absolute value differences				
56	32	10	18	−	10	20	24	17	=	46	12	14	1	add → 456
90	23	128	133		8	10	89	100		82	13	39	33	
24	26	178	200		12	16	178	170		12	10	0	30	
2	0	255	220		4	32	233	112		2	32	22	108	

最近邻分类器

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

最近邻分类器

存储训练数据和标签

最近邻分类器

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

对每一幅测试图像:
找出距离最小的训练图像
返回其对应标签

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

最近邻分类器

Q: 对于 N 幅训练图像, 训练的复杂度?

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

最近邻分类器

Q: 对于 N 幅训练图像, 训练的复杂度?

A: O(1)

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

最近邻分类器

Q:对于 N 幅训练图像, 训练的复杂度?

A: O(1)

Q:对于 N 幅训练图像, 测试复杂度?

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

最近邻分类器

Q:对于 N 幅训练图像, 训练的复杂度?

A: O(1)

Q:对于 N 幅训练图像, 测试复杂度?

A: O(N)

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

最近邻分类器

Q:对于 N 幅训练图像, 训练的复杂度?

A: O(1)

Q:对于 N 幅训练图像, 测试复杂度?

A: O(N)

不理想: 我们能容忍训练缓慢，但是测试需要够快!

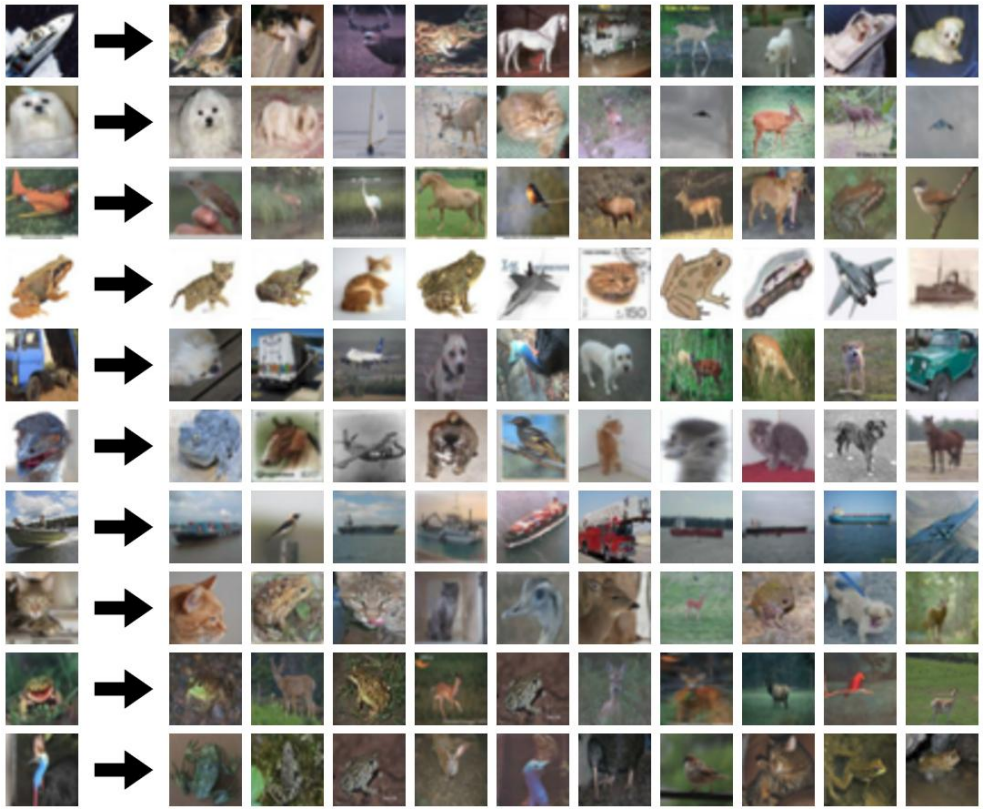
```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

最近邻分类器

有许多方法能够实现快速近邻近似，例如

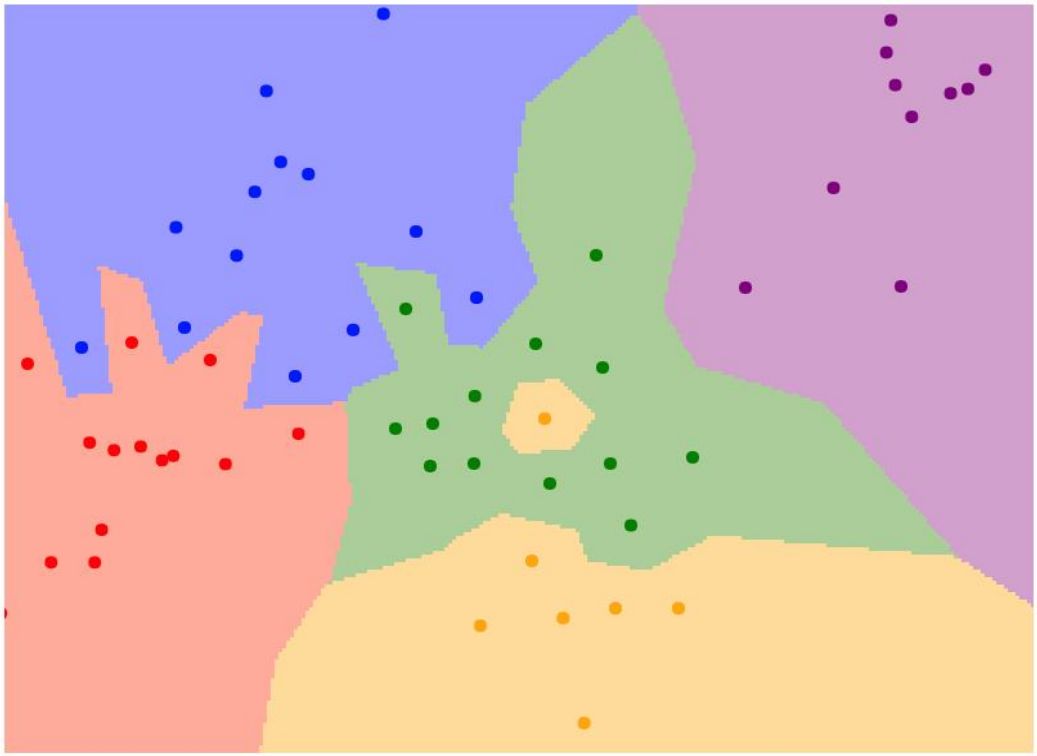
https://github.com/facebookresearch/faiss

这看起来像什么?



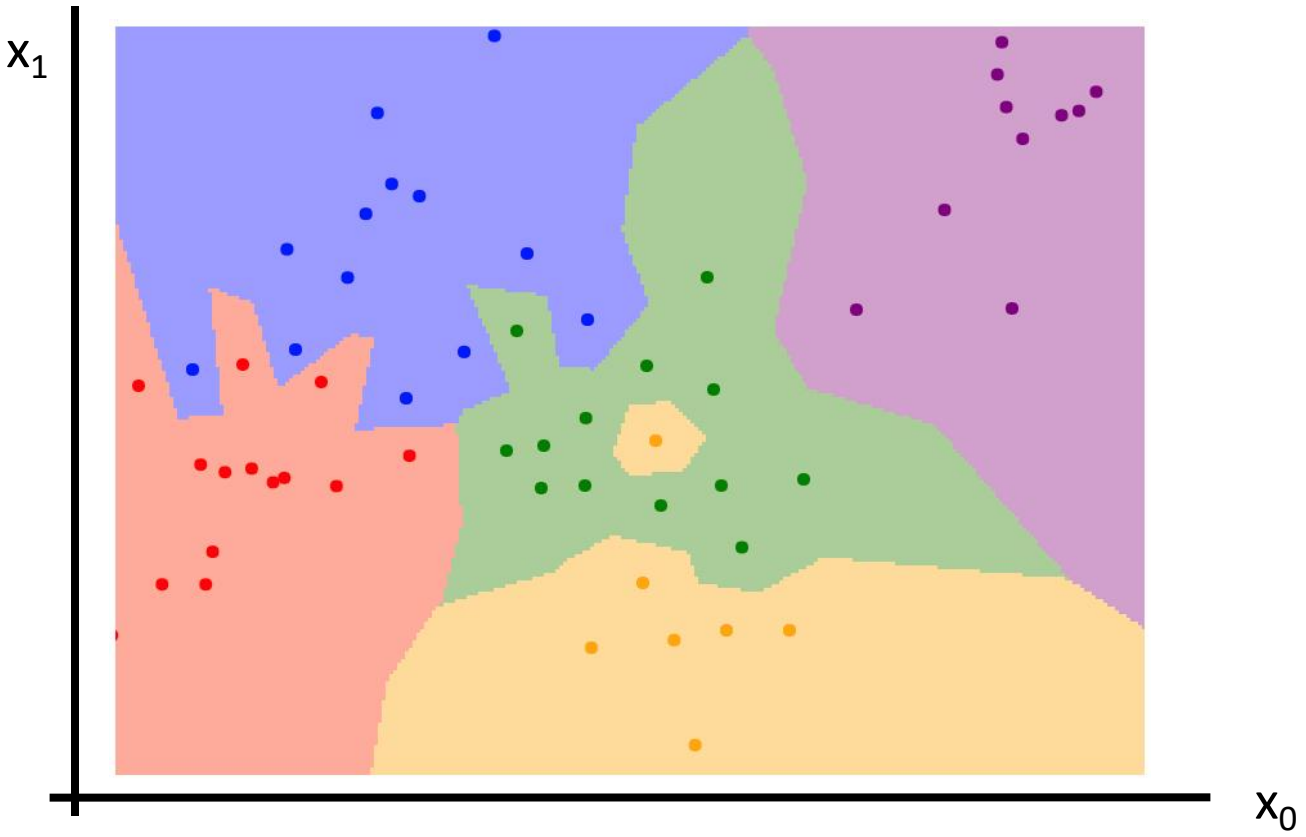
这看起来像什么?

最近邻分类决策边界



最近邻分类决策边界

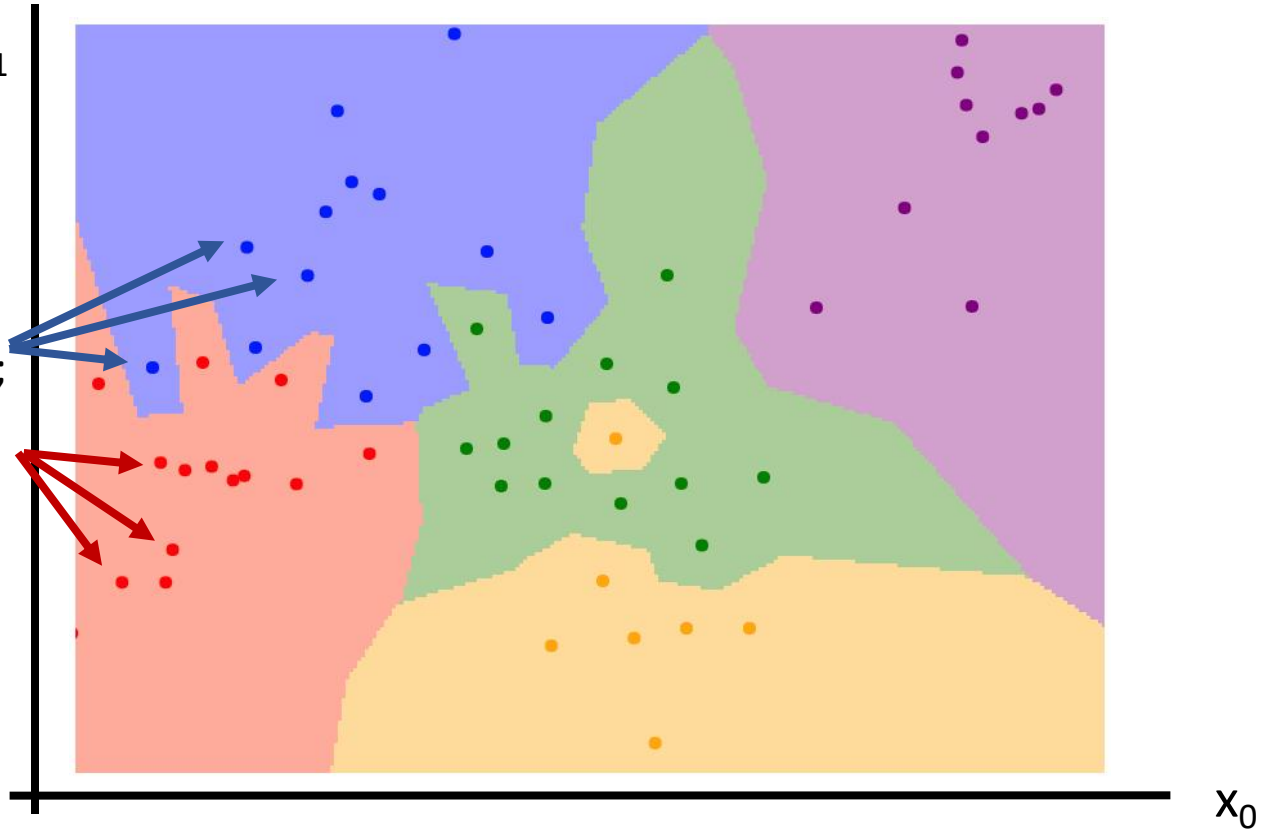
在两个维度上的最近邻



最近邻分类决策边界

在两个维度上的最近邻

点对应训练样本；颜色对应训练样本标签

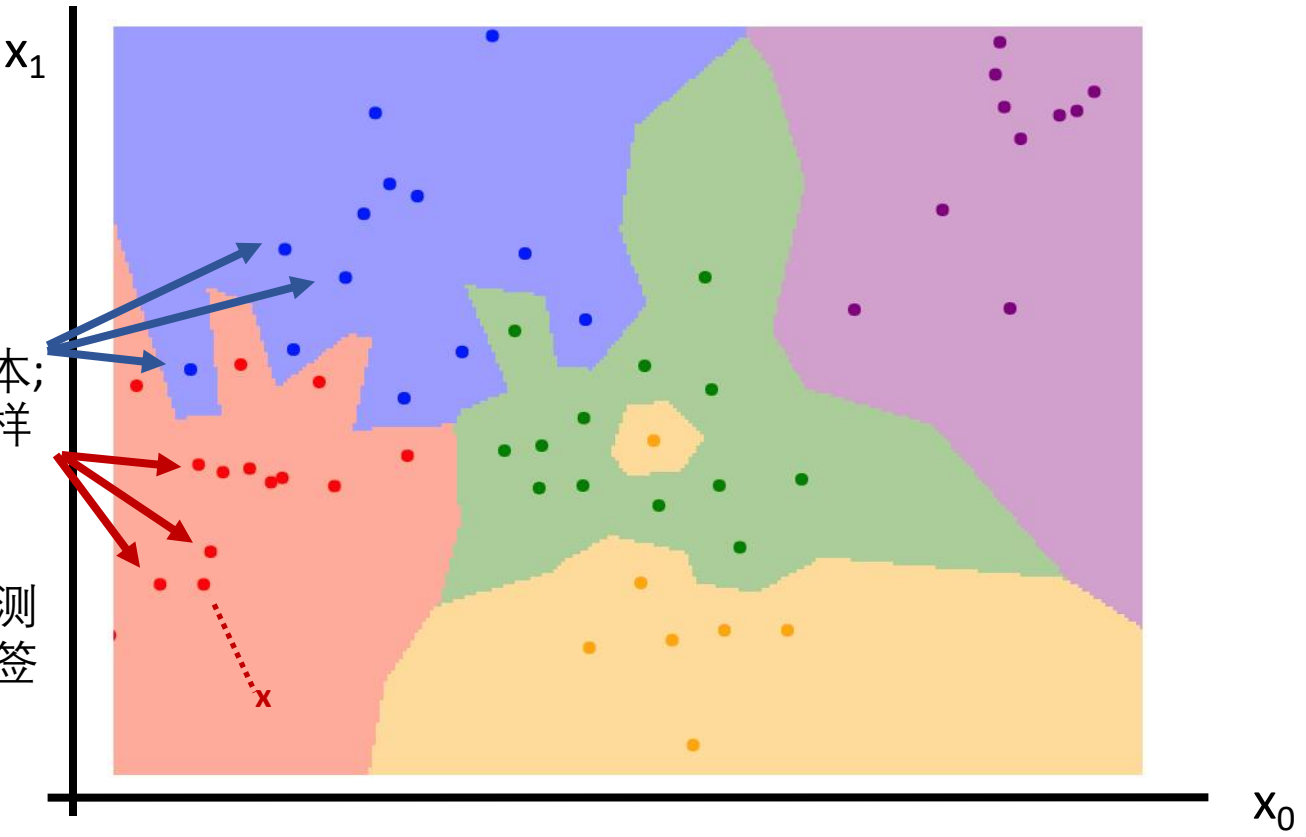


最近邻分类决策边界

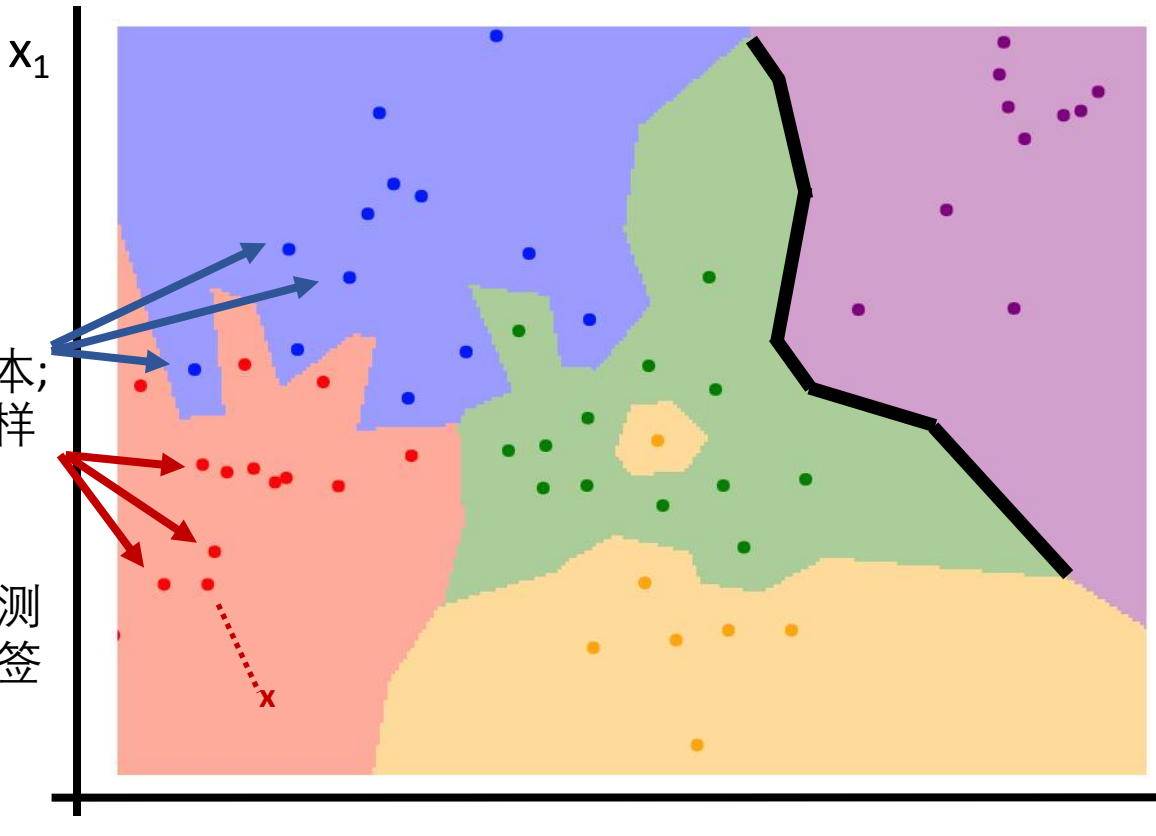
在两个维度上的最近邻

点对应训练样本；颜色对应训练样本标签

背景颜色对应测试图像预测标签



最近邻分类决策边界



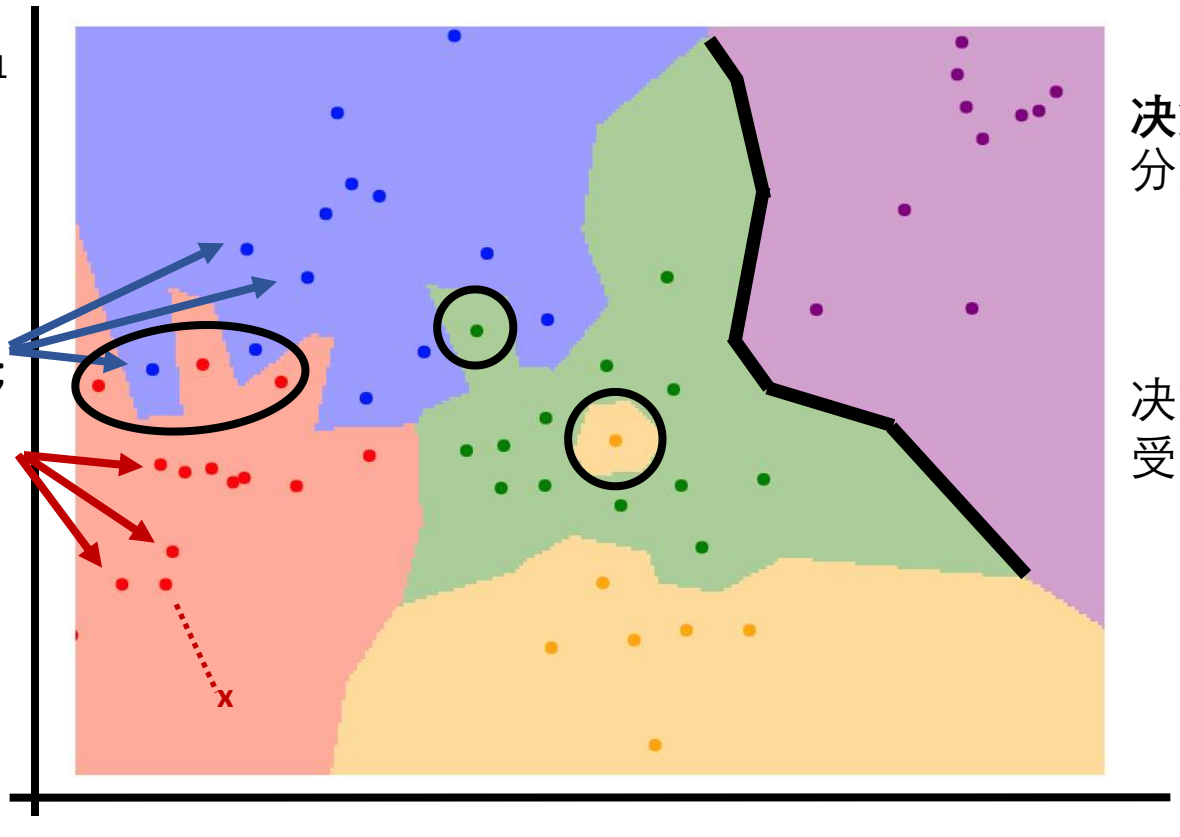
在两个维度上的最近邻

点对应训练样本；颜色对应训练样本标签

背景颜色对应测试图像预测标签

决策边界 是两个分类区域的边界

最近邻分类决策边界



在两个维度上的最近邻

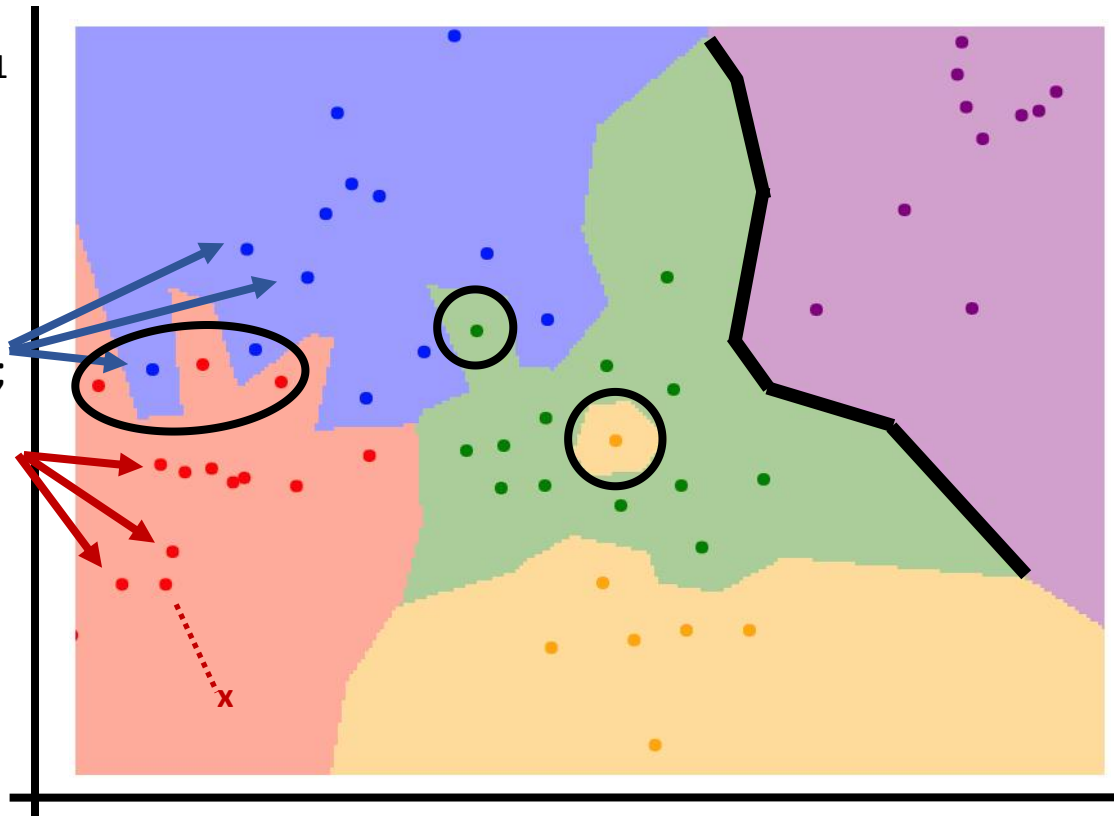
点对应训练样本；颜色对应训练样本标签

背景颜色对应测试图像预测标签

决策边界 是两个分类区域的边界

决策边界存在波动；受离群点影响

最近邻分类决策边界



在两个维度上的最近邻

点对应训练样本；颜色对应训练样本标签

背景颜色对应测试图像预测标签

决策边界 是两个分类区域的边界

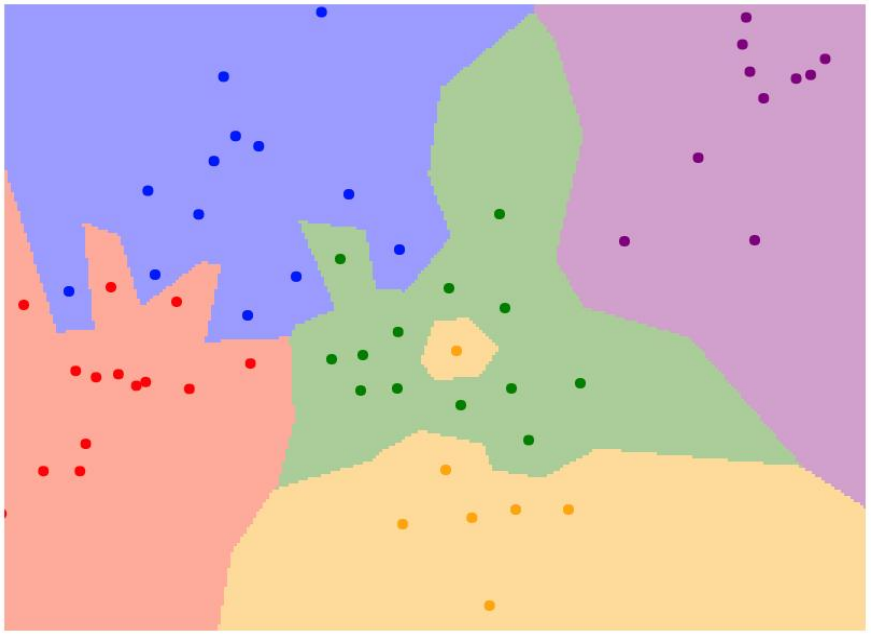
决策边界存在波动；受离群点影响

如何平滑波动的决策边界？
使用更多的邻近点！

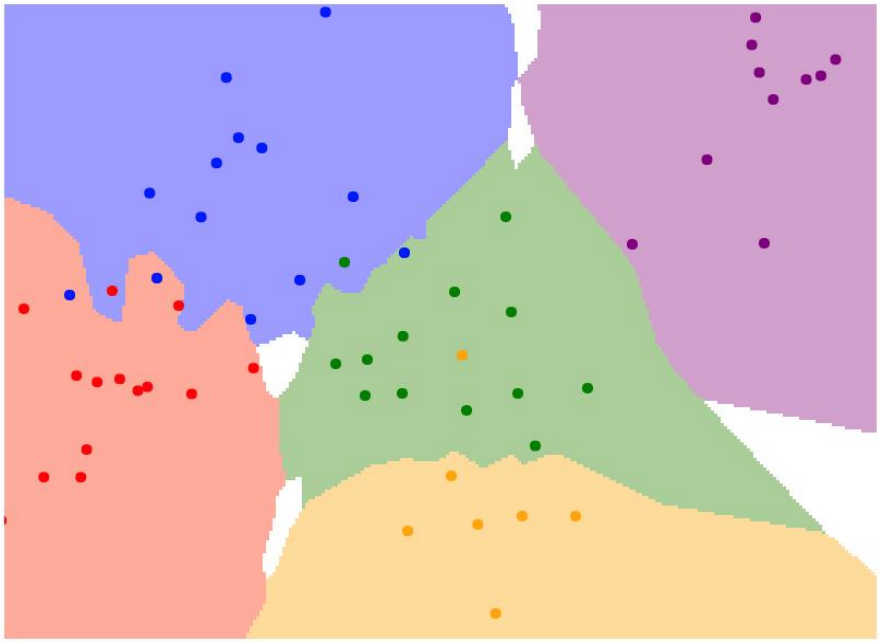
K-近邻分类

与使用最近邻样本标签所不同的是，预测标签为K个最近邻样本标签的大多数

K = 1



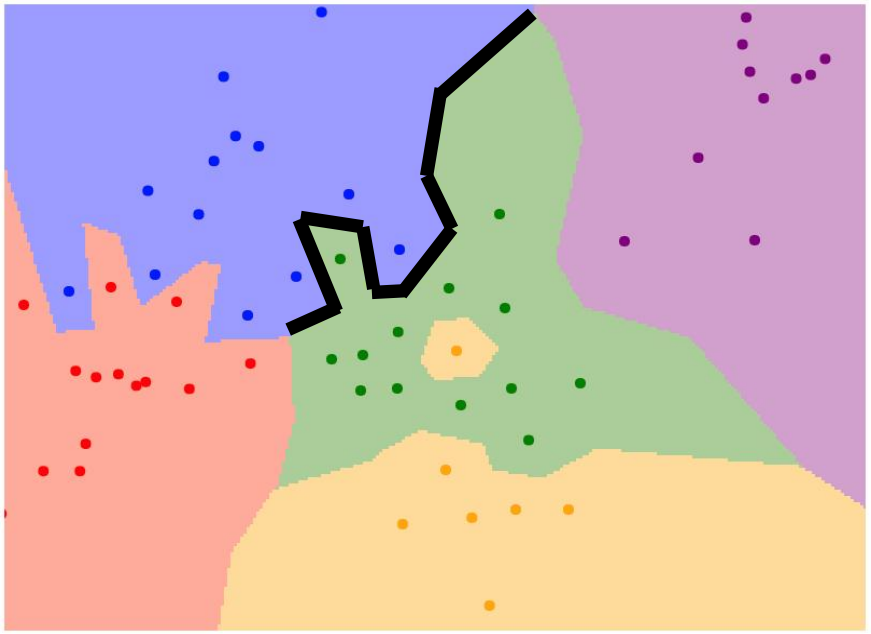
K = 3



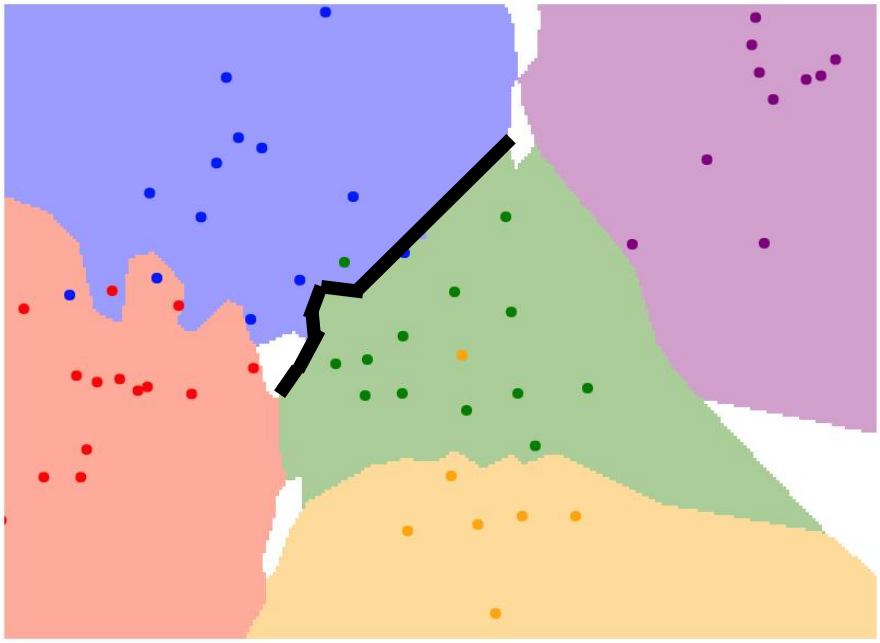
K-近邻分类

与使用最近邻样本标签所不同的是，预测标签为K个最近邻样本标签的大多数

K = 1



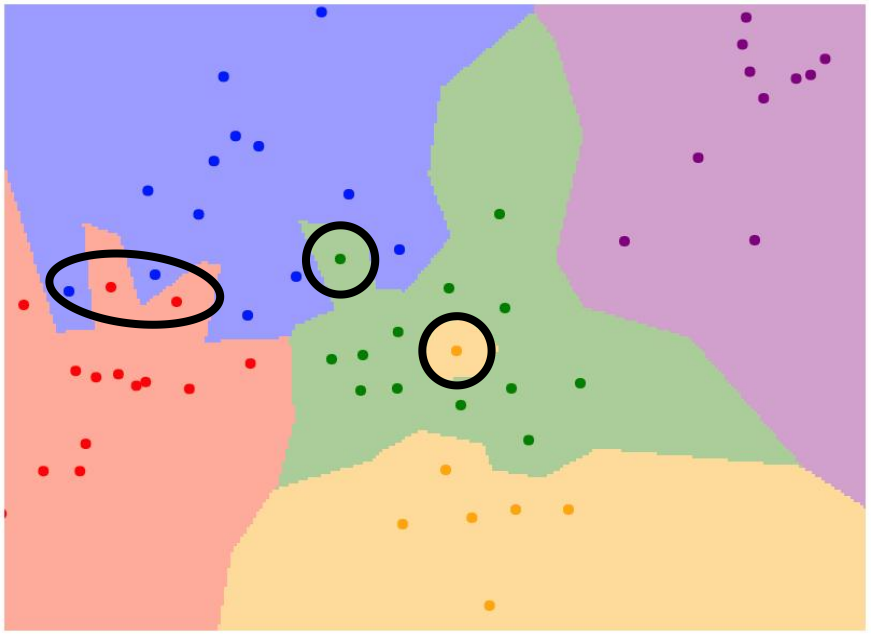
K = 3



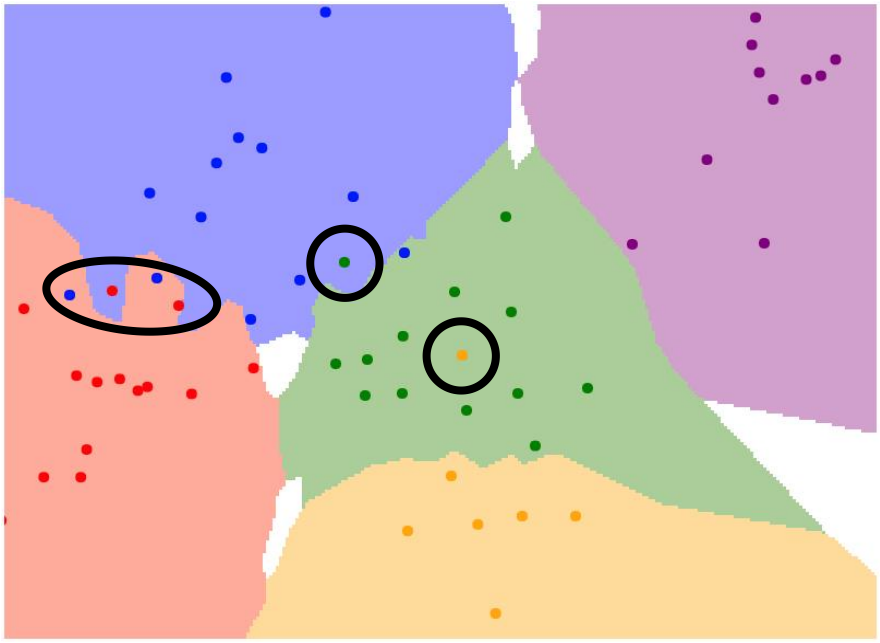
K-近邻分类

使用更多邻域点降低
离群点导致的波动

K = 1



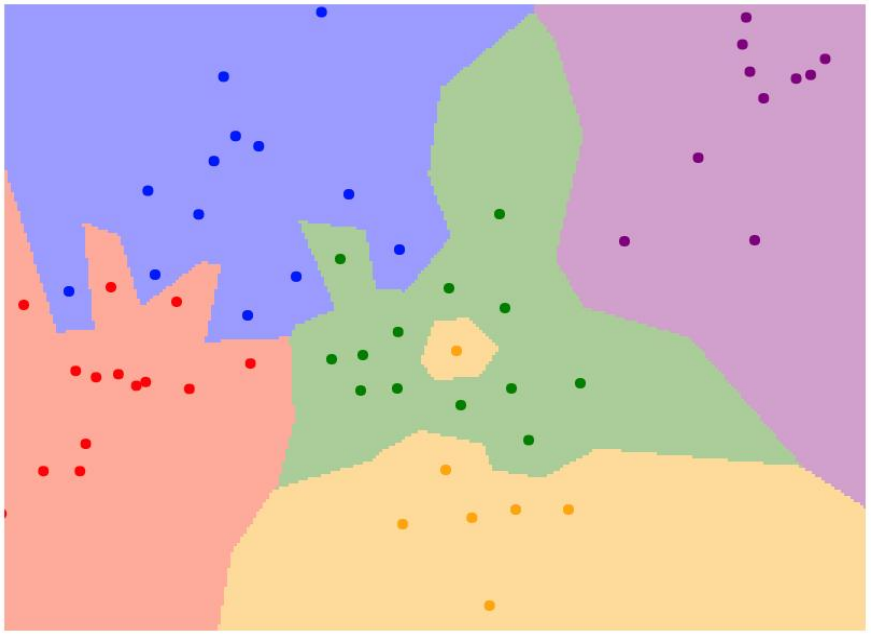
K = 3



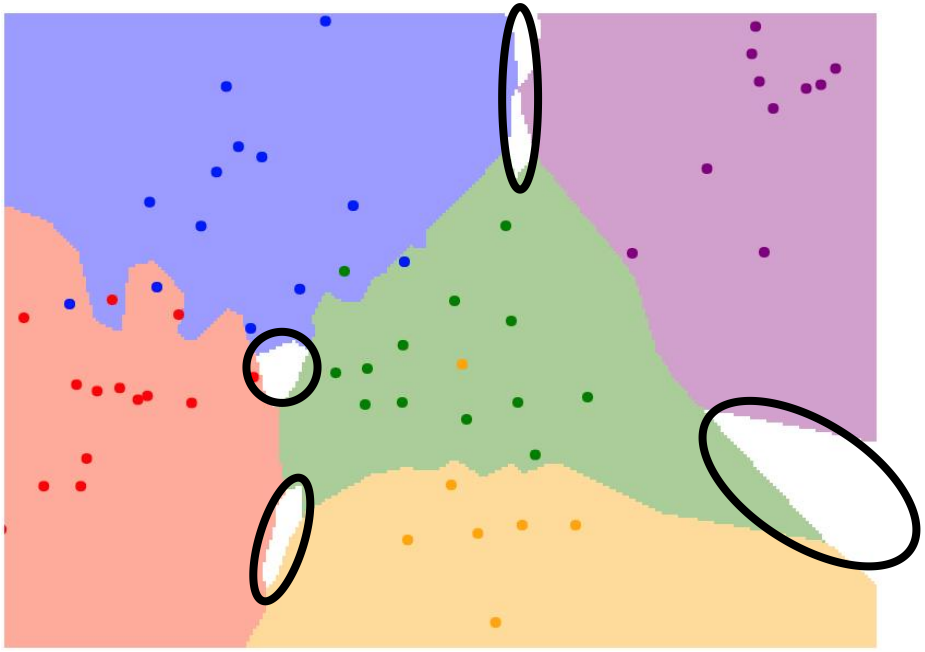
K-近邻分类

当 K > 1 时，类别间存在空隙。需要解决!

K = 1



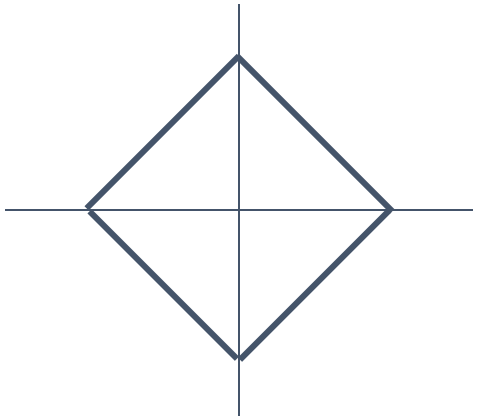
K = 3



K-近邻分类: 距离测度

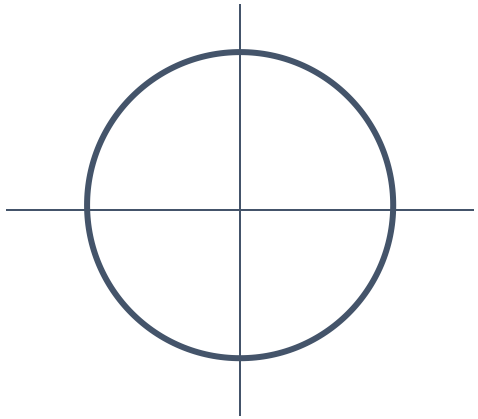
L1 (Manhattan) 距离

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



L2 (Euclidean) 距离

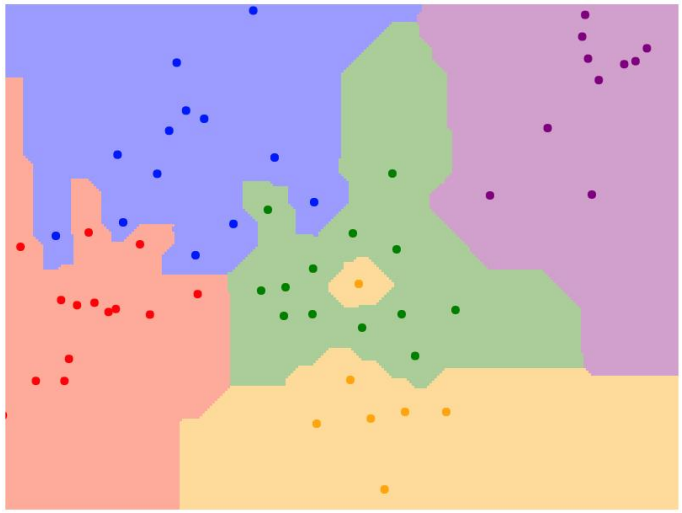
$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$



K-近邻分类: 距离测度

L1 (Manhattan) 距离

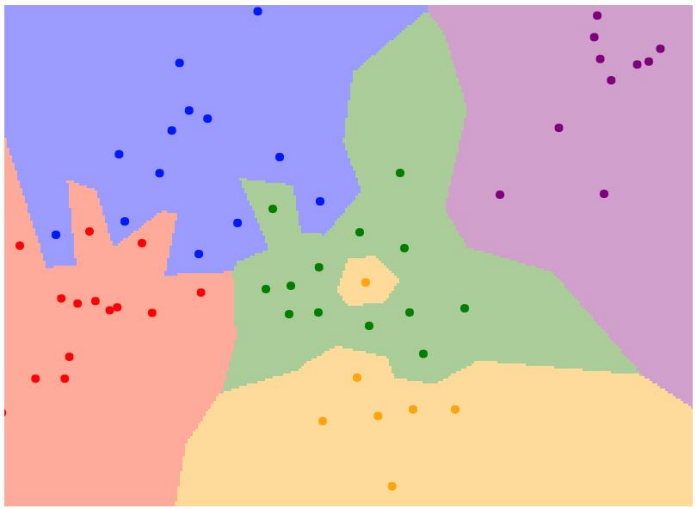
$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



K = 1

L2 (Euclidean) 距离

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$



K = 1

K-近邻分类: 距离测度

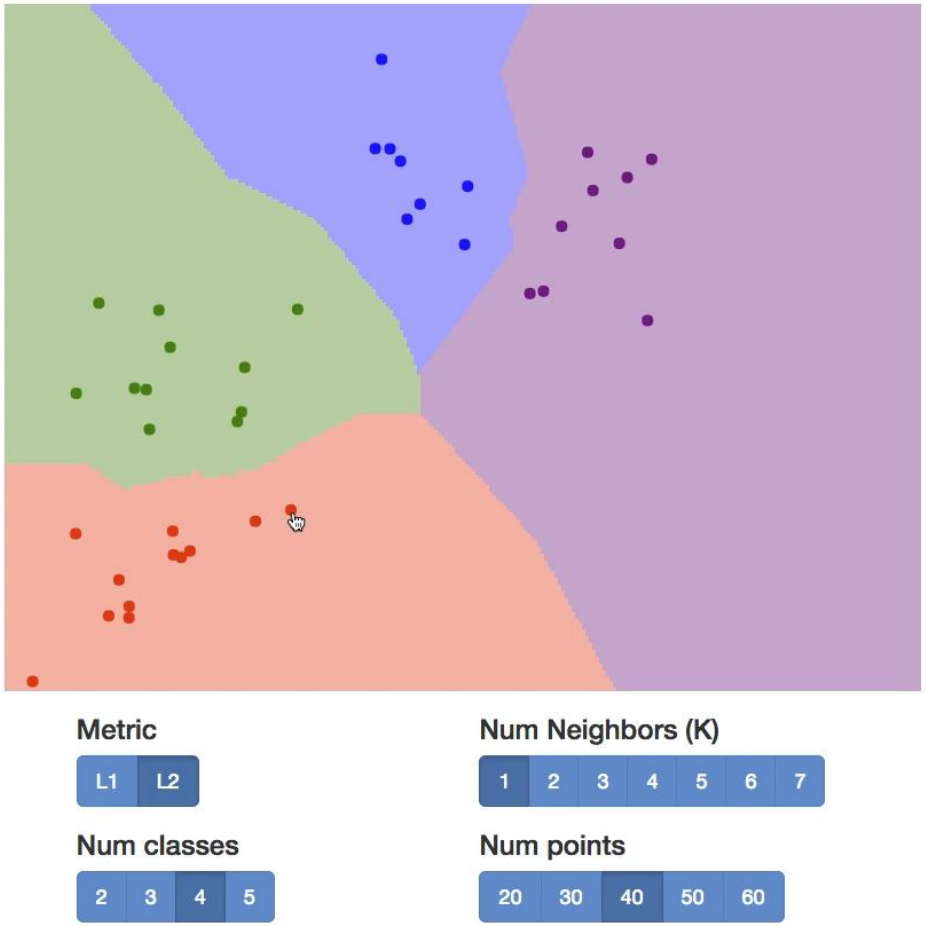
选择正确的距离测度，K-近邻算法能够用于任意类型的数据!

K-近邻分类: 网页示例

可以交互式移动点导致的决策边界变化

可以使用L1 和 L2 测度

可以变化训练点数目，和K的值



http://vision.stanford.edu/teaching/cs231n-demos/knn/

超参数

K的最佳值是什么？
最佳的距离测度是什么？

这些称为**hyperparameters**: 不通过训练数据的学习进行选择；在学习过程之前直接设置

超参数

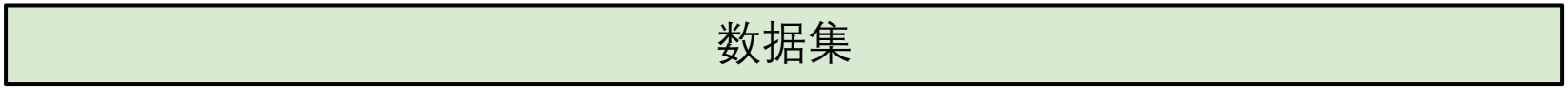
K的最佳值是什么？
最佳的距离测度是什么？

这些称为**hyperparameters**: 不通过训练数据的学习进行选择；在学习过程之前直接设置

超参数是非常问题相关的。通常，需要逐个尝试直到找出适应任务的最优参数。

设置超参数

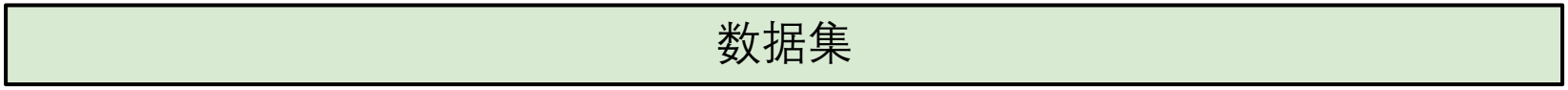
想法 #1: 由所有数据进行超参数的选择



设置超参数

想法 #1: 由所有数据进行超参数的选择

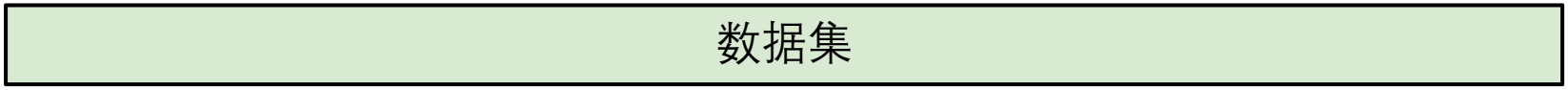
不好: K = 1 在训练集中永远不会出错



设置超参数

想法 #1: 由所有数据进行超参数的选择

不好: K = 1 在训练集中永远不会出错

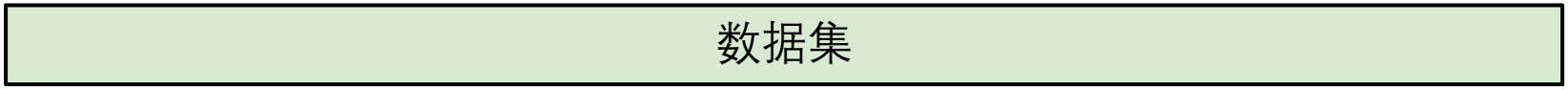


想法 #2: 将数据分为训练集和测试集，根据测试集进行超参数的选择

设置超参数

想法 #1: 由所有数据进行超参数的选择

不好: K = 1 在训练集中永远不会出错



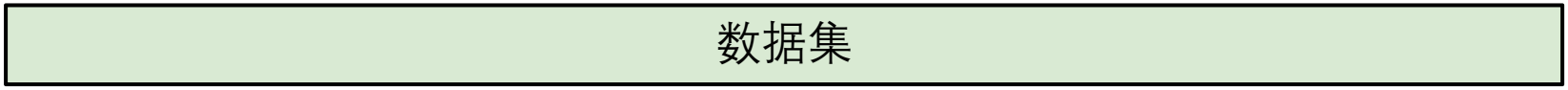
想法 #2: 将数据分为训练集和测试集，根据测试集进行超参数的选择

不好: 不知道算法在新数据上的性能

设置超参数

想法 #1: 由所有数据进行超参数的选择

不好: K = 1 在训练集中永远不会出错



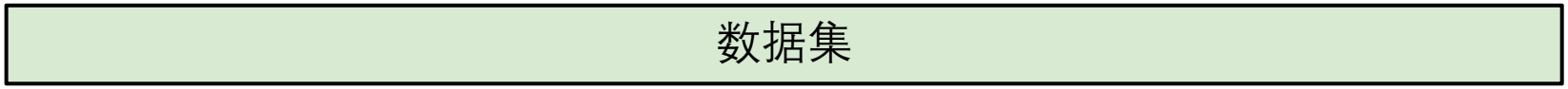
想法 #2: 将数据分为训练集和测试集，根据测试集进行超参数的选择

不好: 不知道算法在新数据上的性能

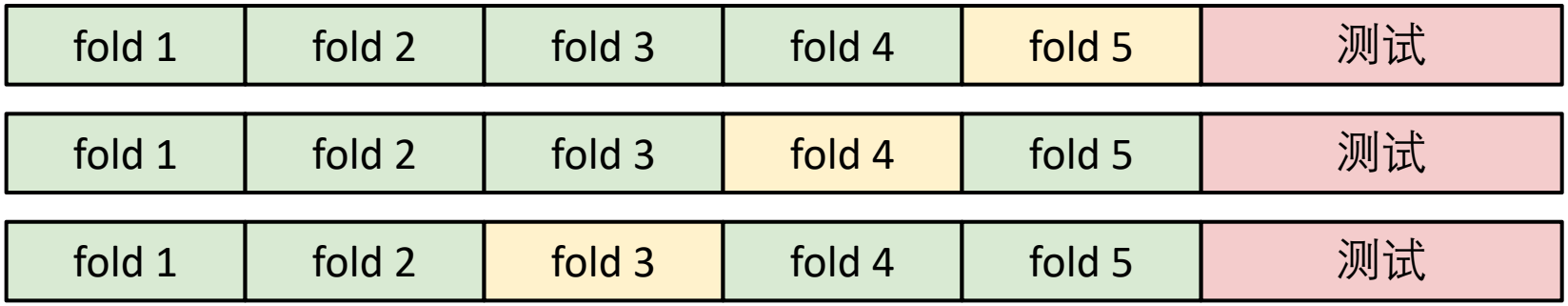
想法 #3: 将数据分为训练，验证和测试；根据验证集和测试集进行超参数的选择

较好!

设置超参数

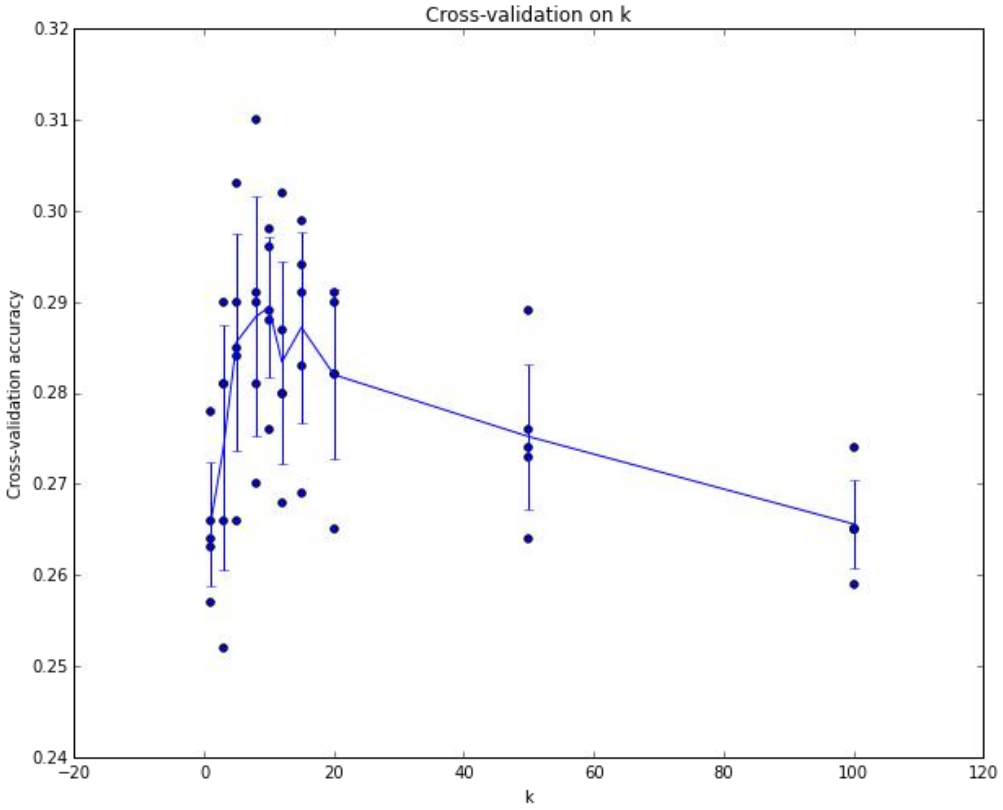


想法 #4: **交叉验证**: 将数据分为多个集合, 依次使用
其中一个集合作为验证集，然后取结果的平均



小数据集时有效，但是深度学习不常用

设置超参数



举例 5-fold 交叉验证随**k**值变化性能**.**

每点：单一测试结果.

线段经过均值，长度表示标准差

(从结果看k ~ 7 性能最佳)

K-近邻 很少用于图像质量不高情况

- 测试慢
- 距离测度没有体现像素信息

原始

带黑框

有平移

颜色变化

(所有 3 幅图像和原始图像有相同的 L2 距离)

最近邻分类适用于ConvNet特征分类!

Devlin et al, “Exploring Nearest Neighbor Approaches for Image Captioning”, 2015

本章内容

- 5.1.为什么需要学习
- 5.2.最近邻方法
- 5.3.线性分类器
- 5.4.损失函数与正则化
- 5.5.最优化

神经网络

线性分类器

CIFAR10数据集

50,000 训练图像
每张图像大小 **32x32x3**

10,000 测试图像.

参数化方法

图像

32x32x3 矩阵
(共3072个)

f(**x**,**W**)

W
参数
或 权重

10 个数字对应各类别的响应分值

参数化方法：线性分类器

$$f(x,W) = Wx$$

图像

32x32x3 矩阵
(共3072个)

f(**x**,**W**)

10 个数字对应各类别的响应分值

W
参数
或 权重

参数化方法：线性分类器

图像

32x32x3 矩阵
(共3072个)

f(x,W) = Wx

(10,) (10, 3072) (3072,)

f(**x**,**W**)

10 个数字对应各类别的响应分值

W
参数
或 权重

参数化方法：线性分类器

图像

32x32x3 矩阵
(共3072个)

f(x,W) = Wx + b

(10,) (10, 3072) (3072,) (10,)

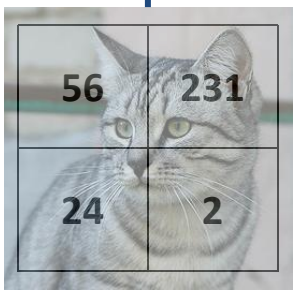
f(**x**,**W**)

10 个数字对应各类别的响应分值

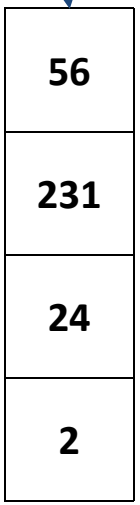
W
参数
或 权重

举例2x2 图像, 3 个类别 (猫/狗/船)

将像素拉伸为列向量



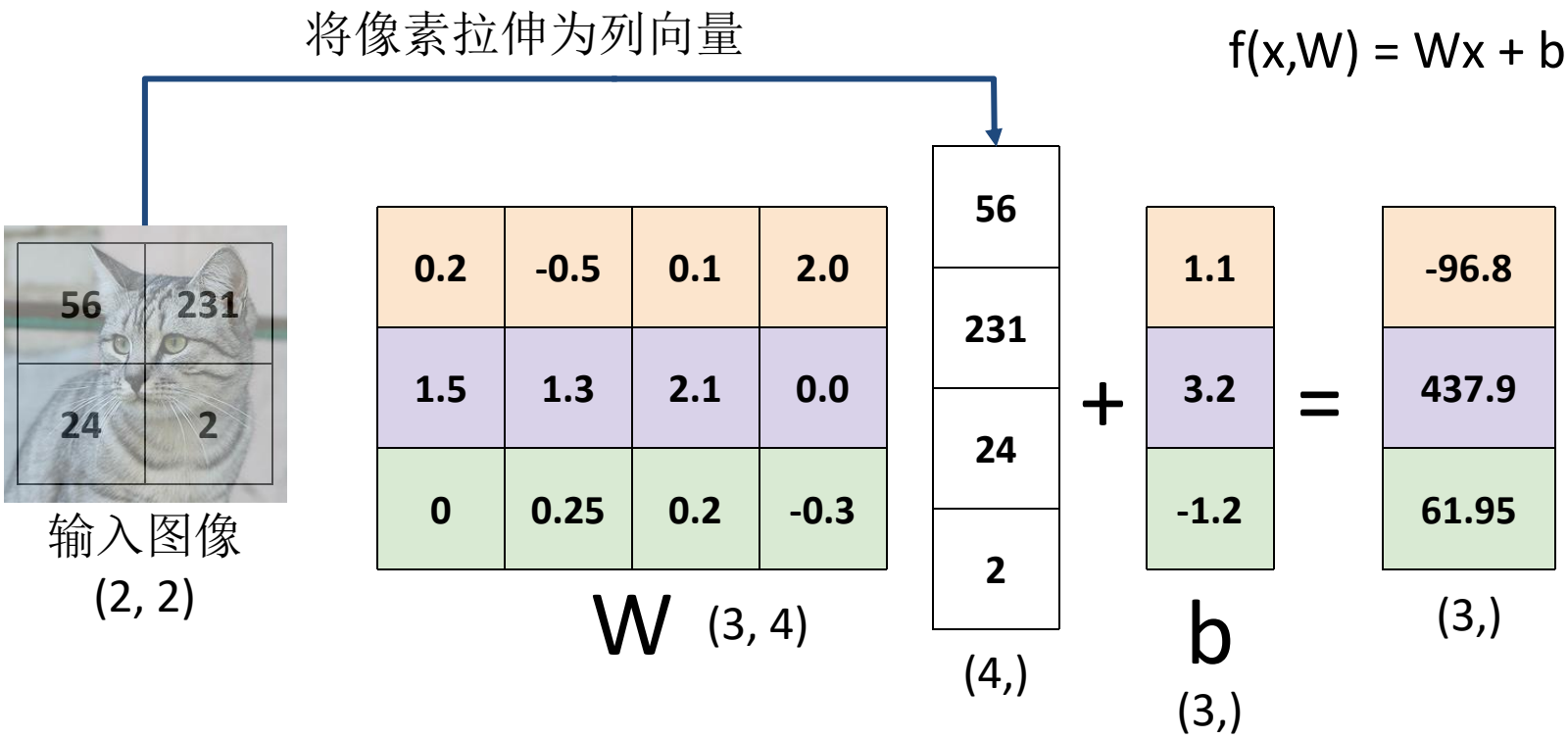
输入图像
(2, 2)



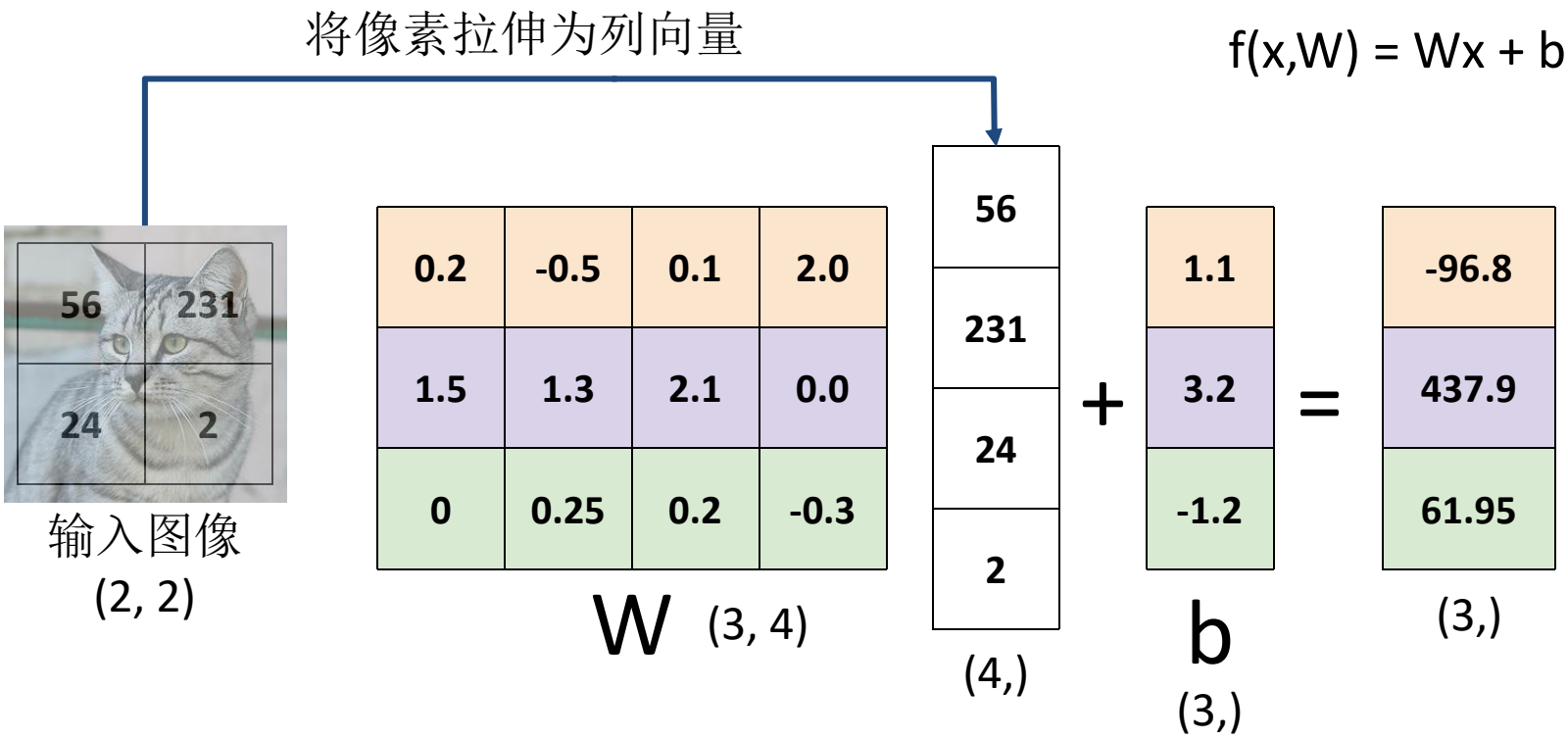
(4,)

$f(x,W) = Wx + b$

举例2x2 图像, 3 个类别 (猫/狗/船)



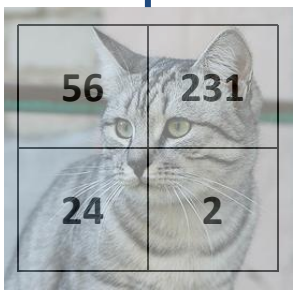
线性分类器：代数角度


将像素拉伸为列向量
f(x,W) = Wx + b
56
231
24
2
输入图像
(2, 2)
0.2
-0.5
0.1
2.0
1.5
1.3
2.1
0.0
0
0.25
0.2
-0.3
W
(3, 4)
56
231
24
2
(4,)
+
1.1
3.2
-1.2
b
(3,)
=
-96.8
437.9
61.95
(3,)

线性分类器：偏置小技巧

增加一个维度到数据向量中；偏置对应权重向量的最后一个

将像素拉伸为列向量



输入图像
(2, 2)

0.2	-0.5	0.1	2.0	1.1
1.5	1.3	2.1	0.0	3.2
0	0.25	0.2	-0.3	-1.2

W (3, 5)

56
231
24
2
1

(5,)

=

-96.8
437.9
61.95

(3,)

线性分类器： 预测模型为线性的!

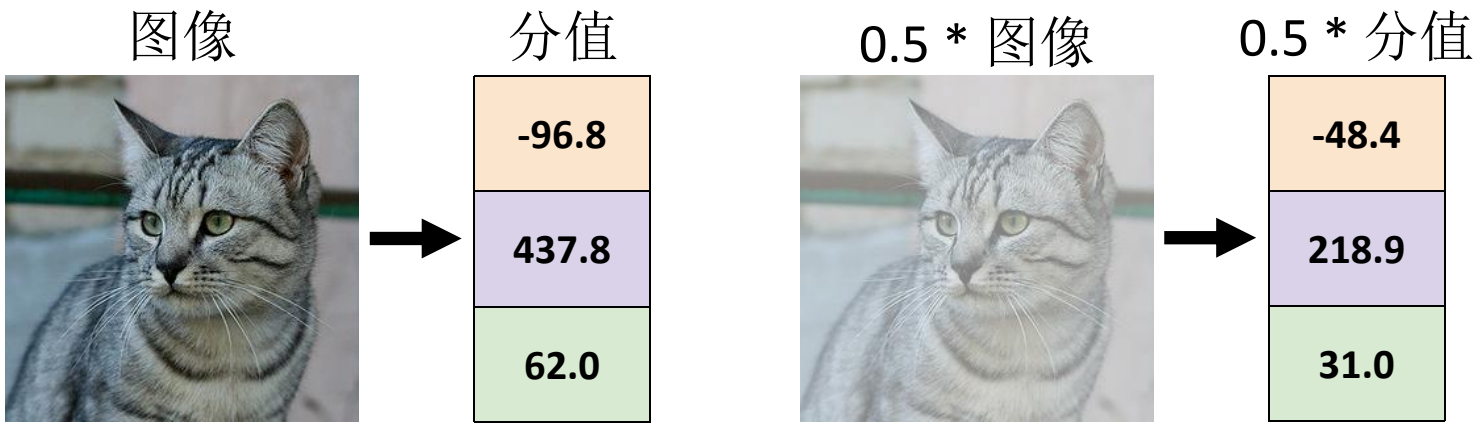
$f(x, W) = Wx$ (忽略偏置)

$f(cx, W) = W(cx) = c * f(x, W)$

线性分类器：预测模型为线性的!

$$f(x, W) = Wx \quad \text{(忽略偏置)}$$

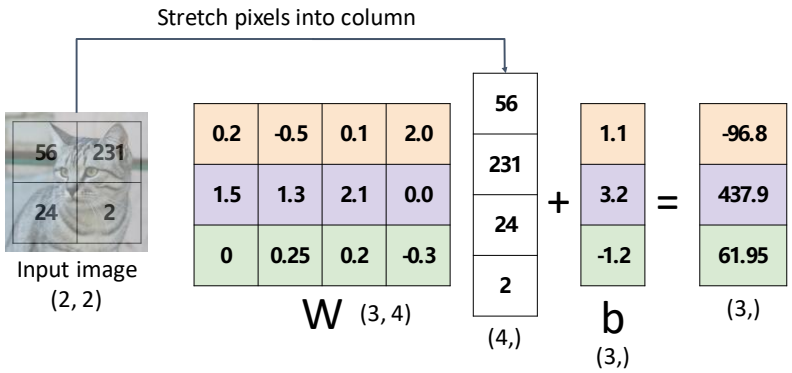
$$f(cx, W) = W(cx) = c * f(x, W)$$



线性分类器的解释

代数角度

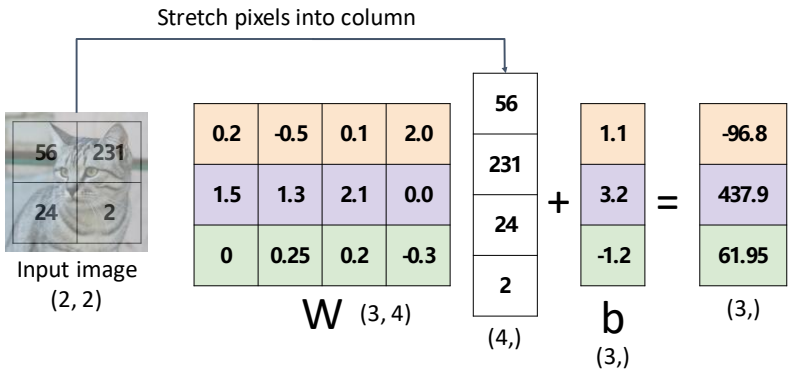
$$f(x,W) = Wx + b$$

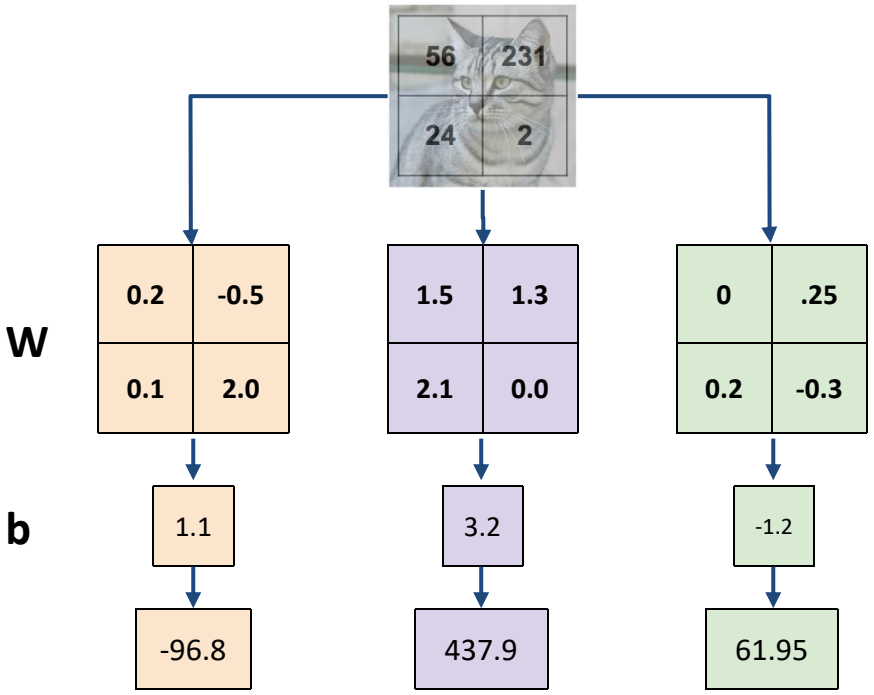


线性分类器的解释

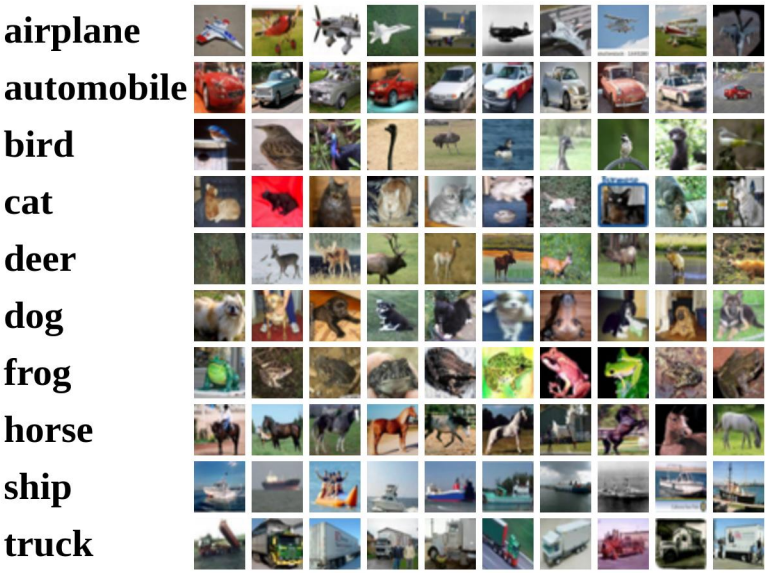
代数角度

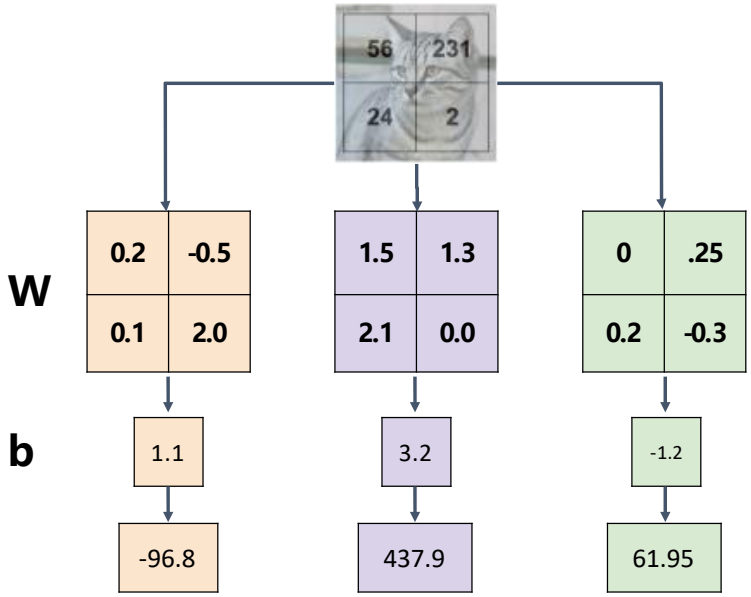
$$f(x,W) = Wx + b$$



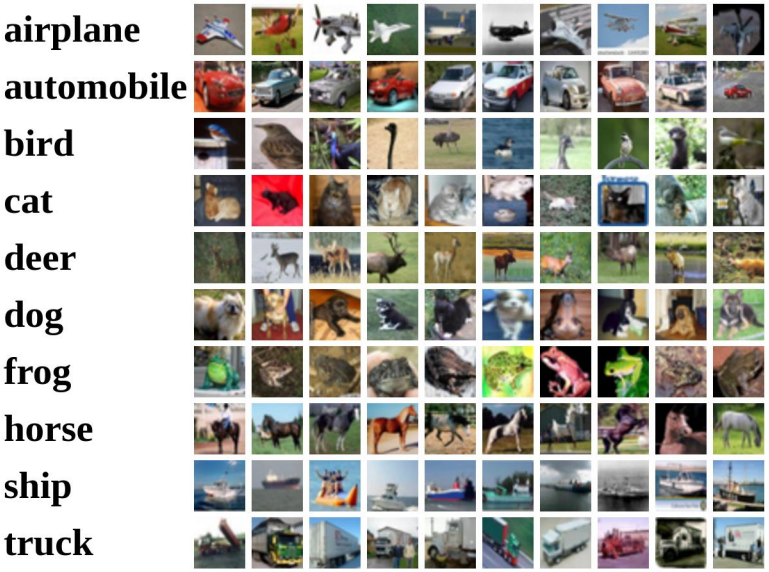


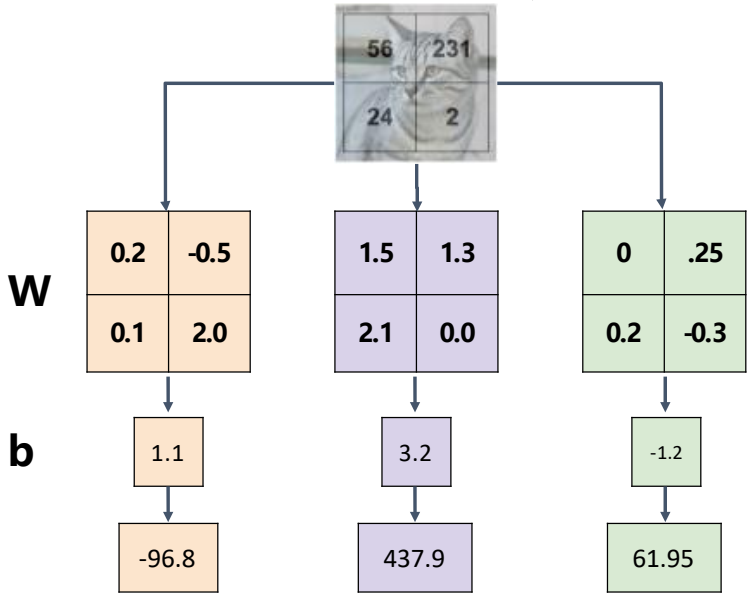
线性分类器的解释


airplane
automobile
bird
cat
deer
dog
frog
horse
ship
truck


56
231
24
2
W
0.2
-0.5
0.1
2.0
1.5
1.3
2.1
0.0
0
.25
0.2
-0.3
b
1.1
3.2
-1.2
-96.8
437.9
61.95

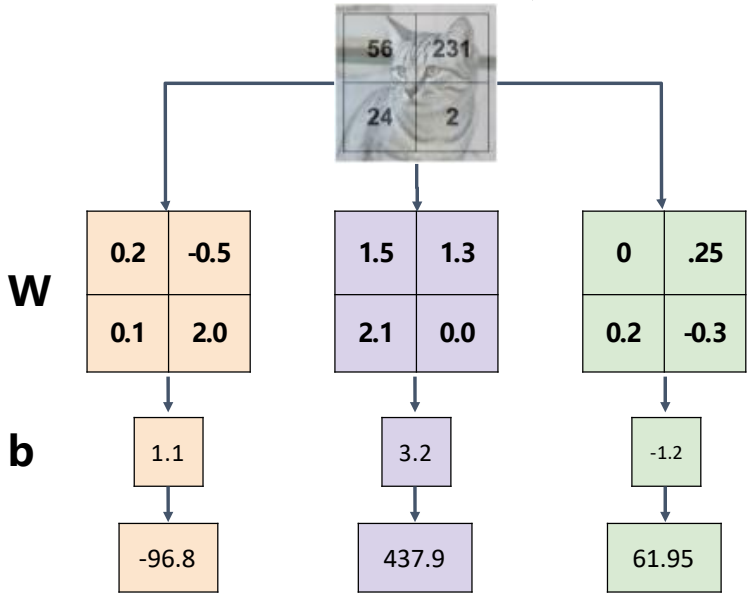
线性分类器的解释：可视化角度





线性分类器的解释：可视化角度

线性分类器每个类别
有一个模板

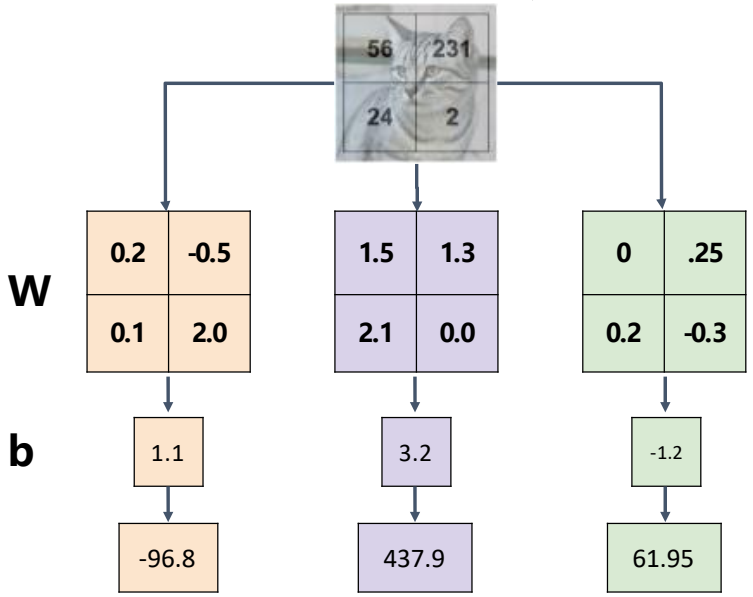


线性分类器的解释：可视化角度

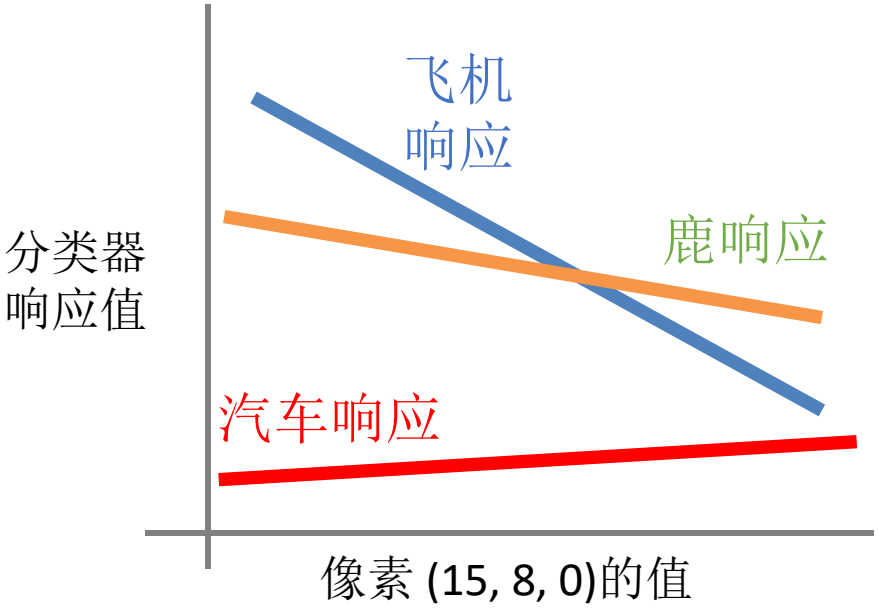
线性分类器每个类别
有一个模板

单一模板无法建模多模式数据

例. 马的模板有两个头!



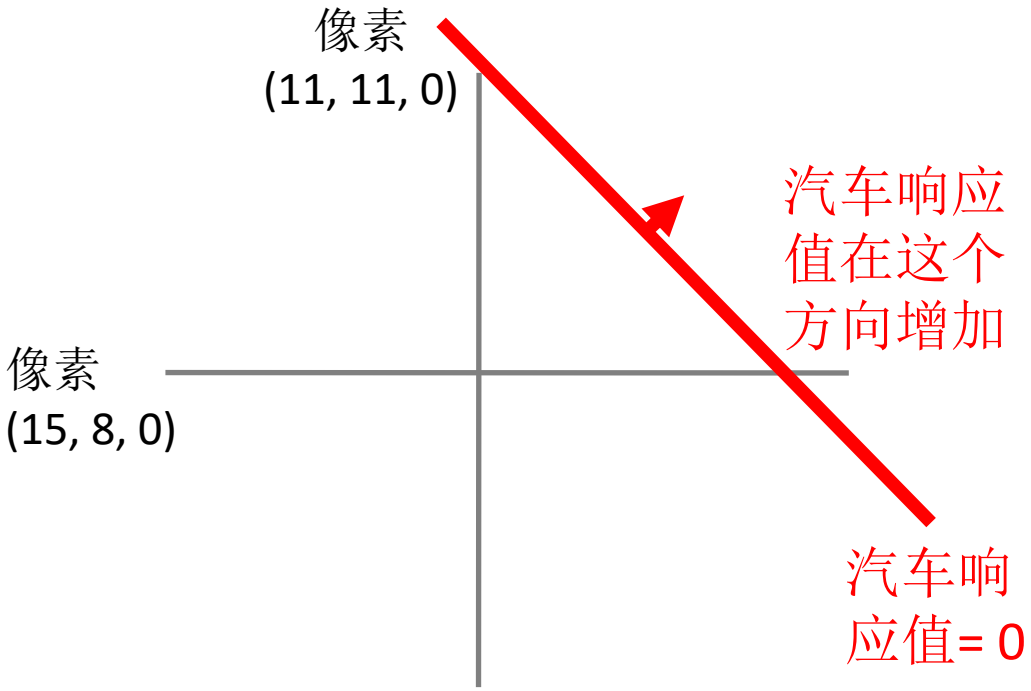
线性分类器的解释：几何角度



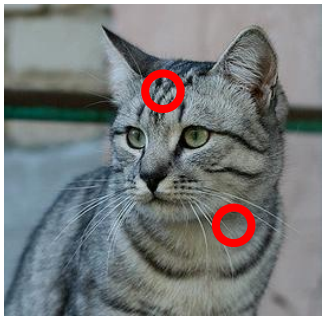
$$f(x,W) = Wx + b$$

32x32x3 矩阵
(共3072个)

线性分类器的解释：几何角度

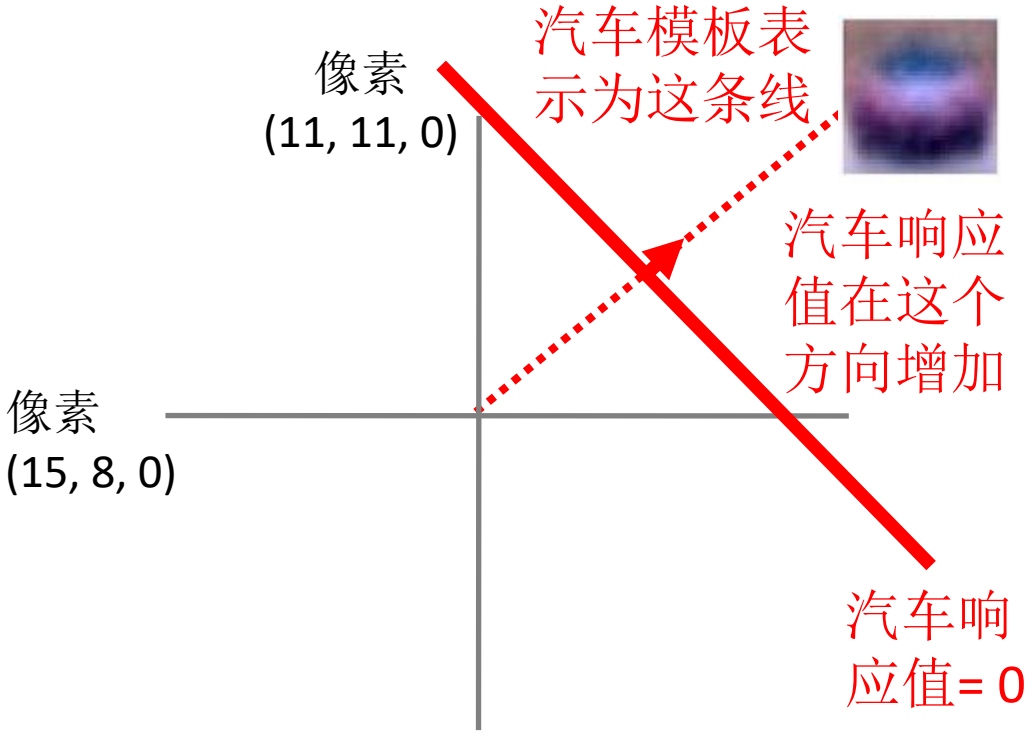


$$f(x,W) = Wx + b$$

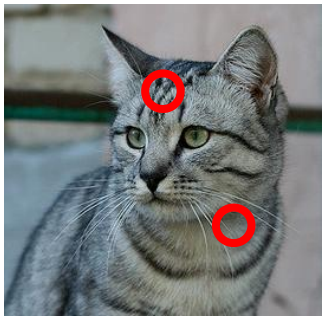


32x32x3 矩阵
(共3072个)

线性分类器的解释：几何角度

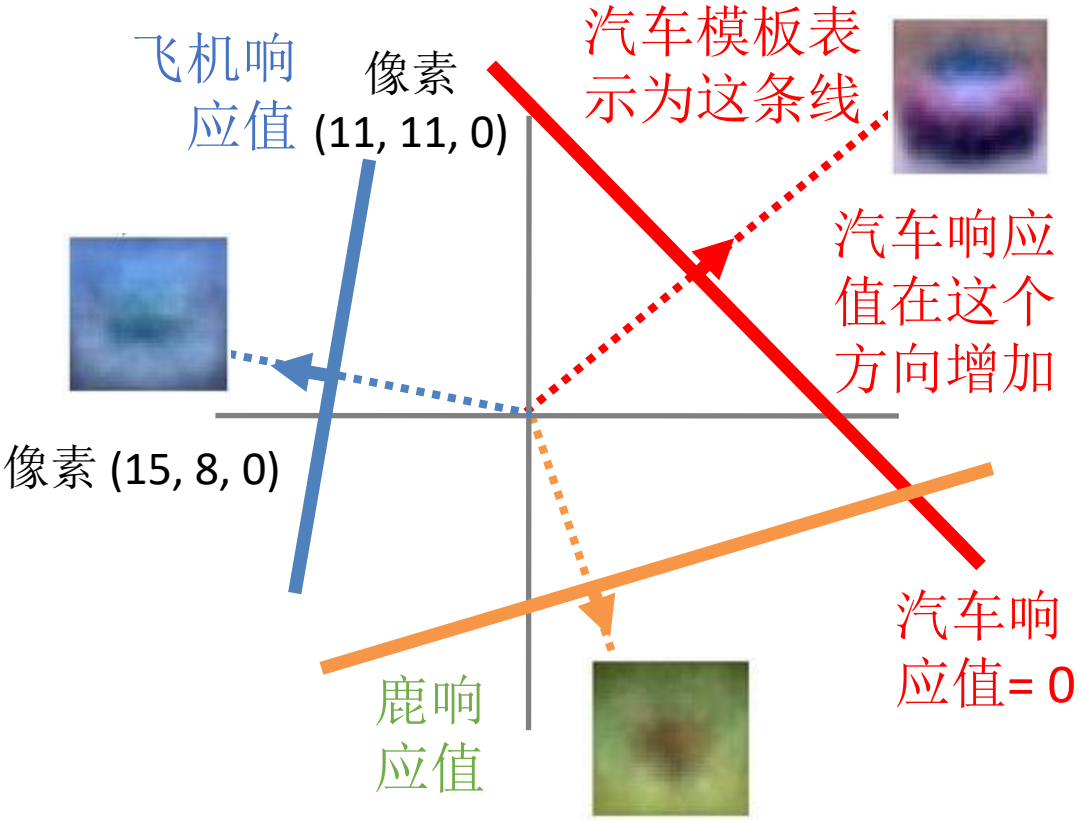


$$f(x,W) = Wx + b$$

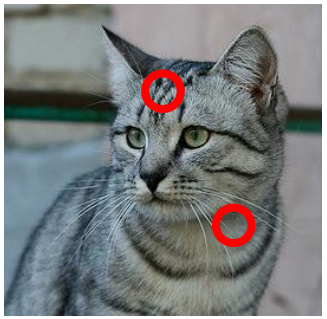


32x32x3 矩阵
(共3072个)

线性分类器的解释：几何角度

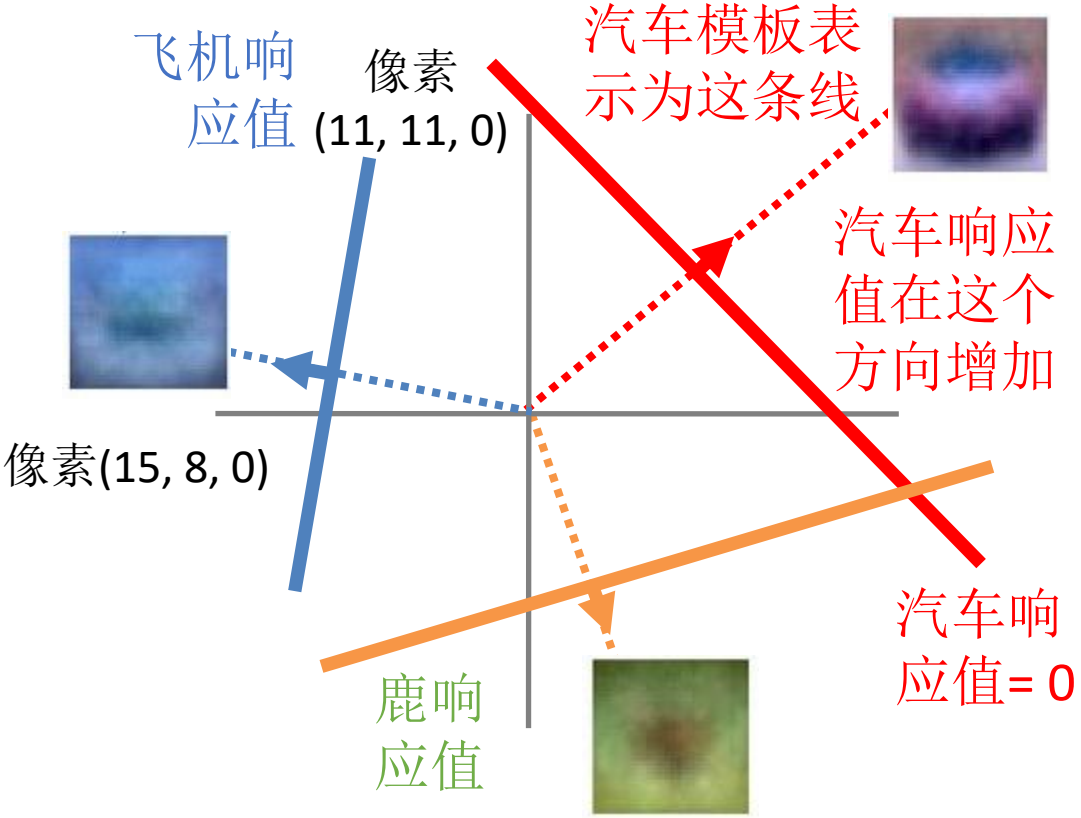


$$f(x,W) = Wx + b$$

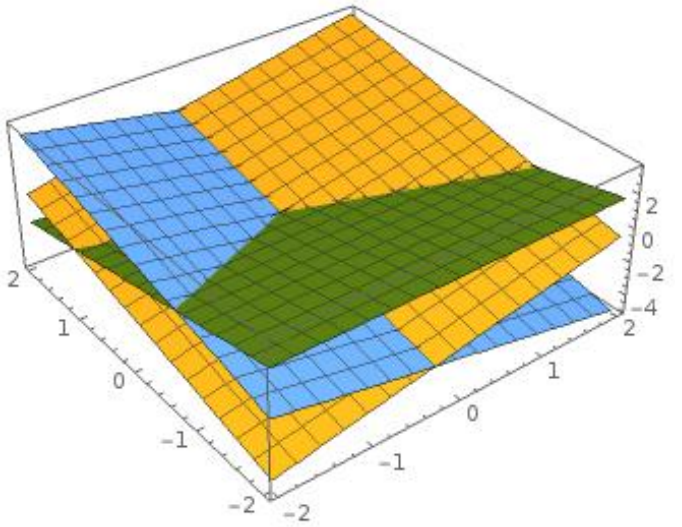


32x32x3 矩阵
(共3072个)

线性分类器的解释：几何角度



分割高维空间的超平面

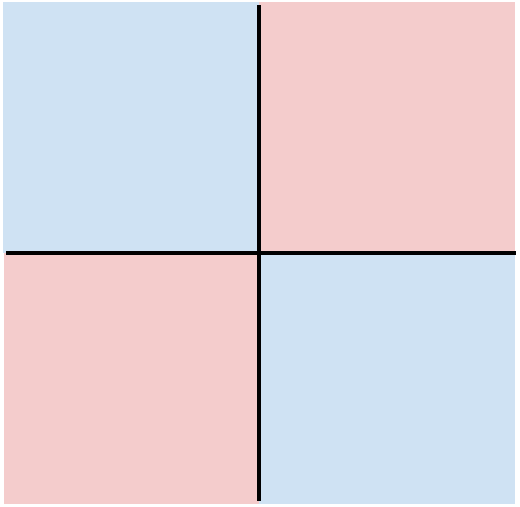


Plot created using Wolfram Cloud

线性分类器的困难情况

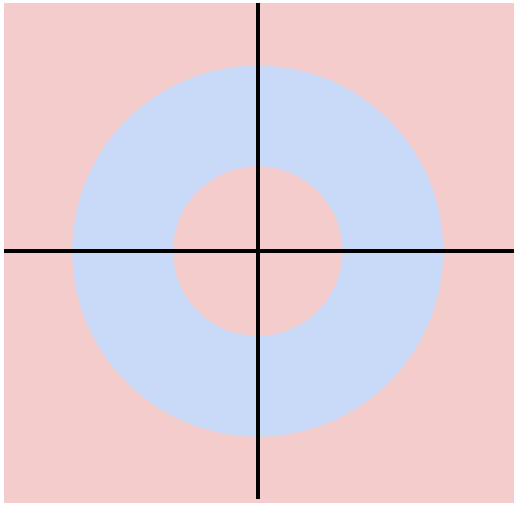
类别 1:
一三象限

类别 2:
二四象限



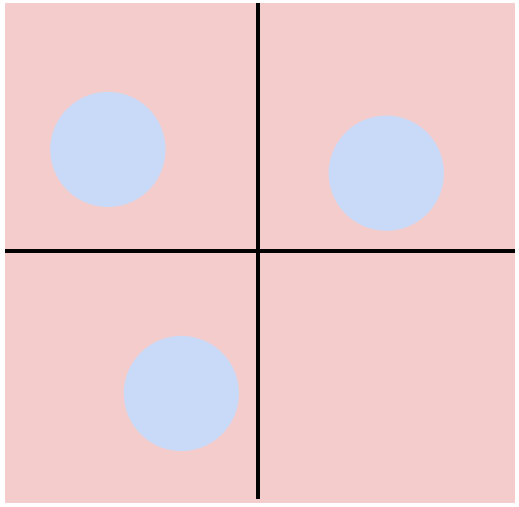
类别 1:
1 <= L2 范数 <= 2

类别 2:
其他区域



类别 1:
三个模式

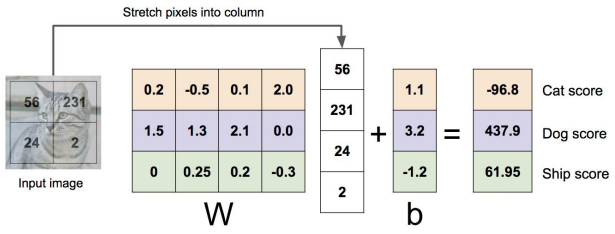
类别 2:
其他区域



线性分类器: 三个角度

代数角度

$$f(x,W) = Wx$$

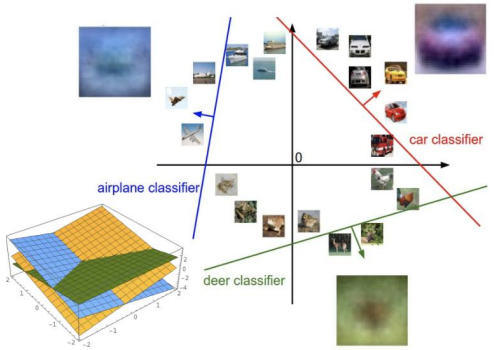


可视化角度

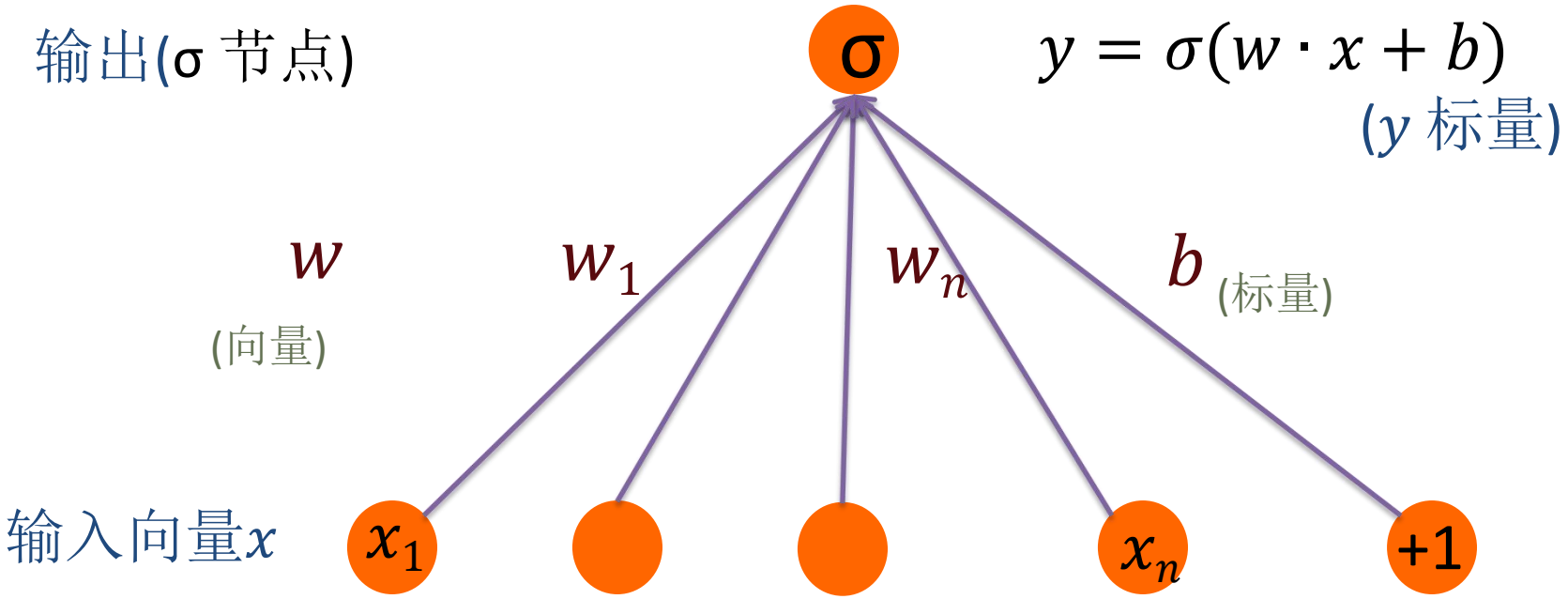
每类一个模板

几何角度

超平面切割

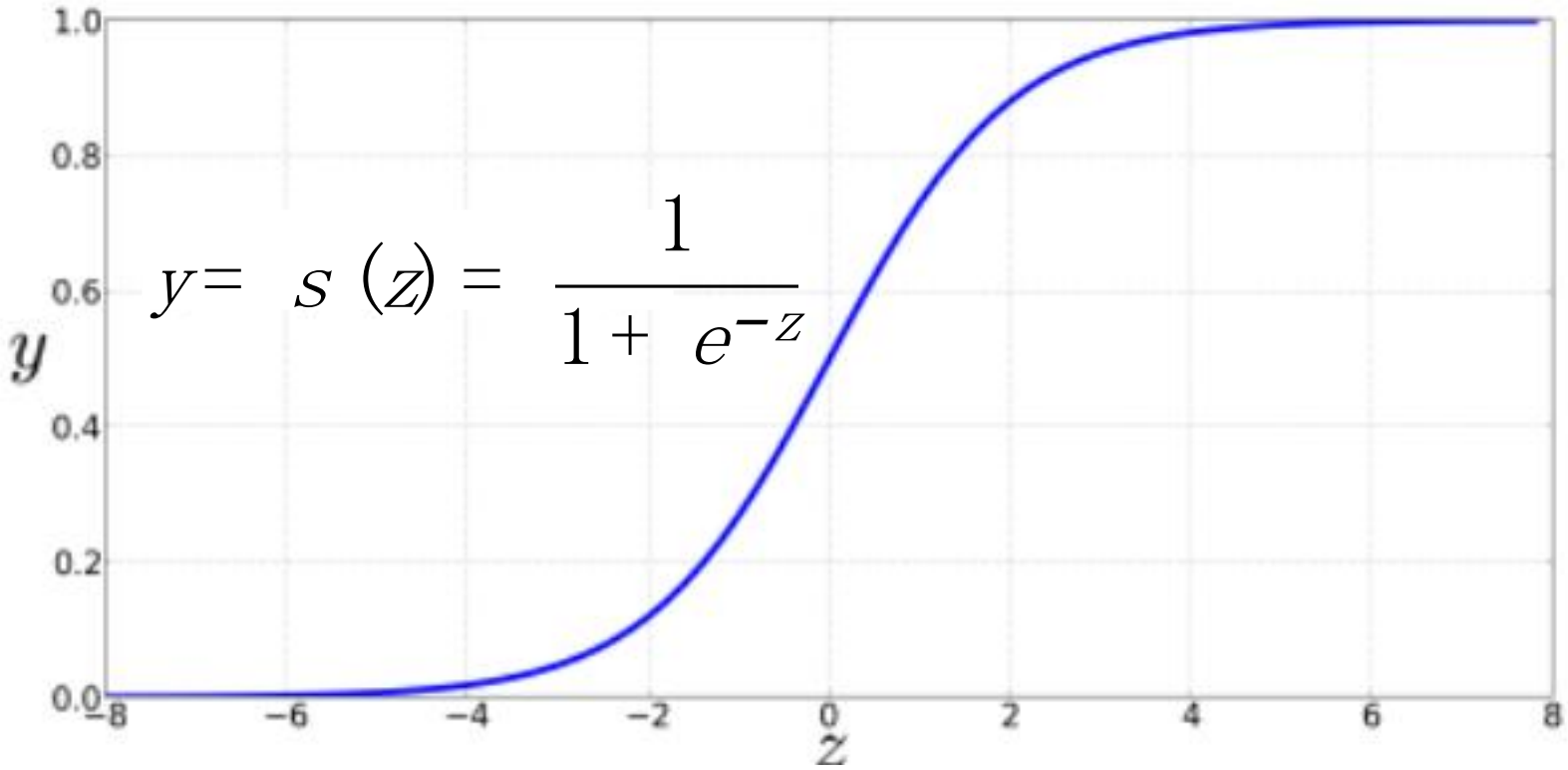


二元Logistics回归



Logistics回归： sigmoid 函数

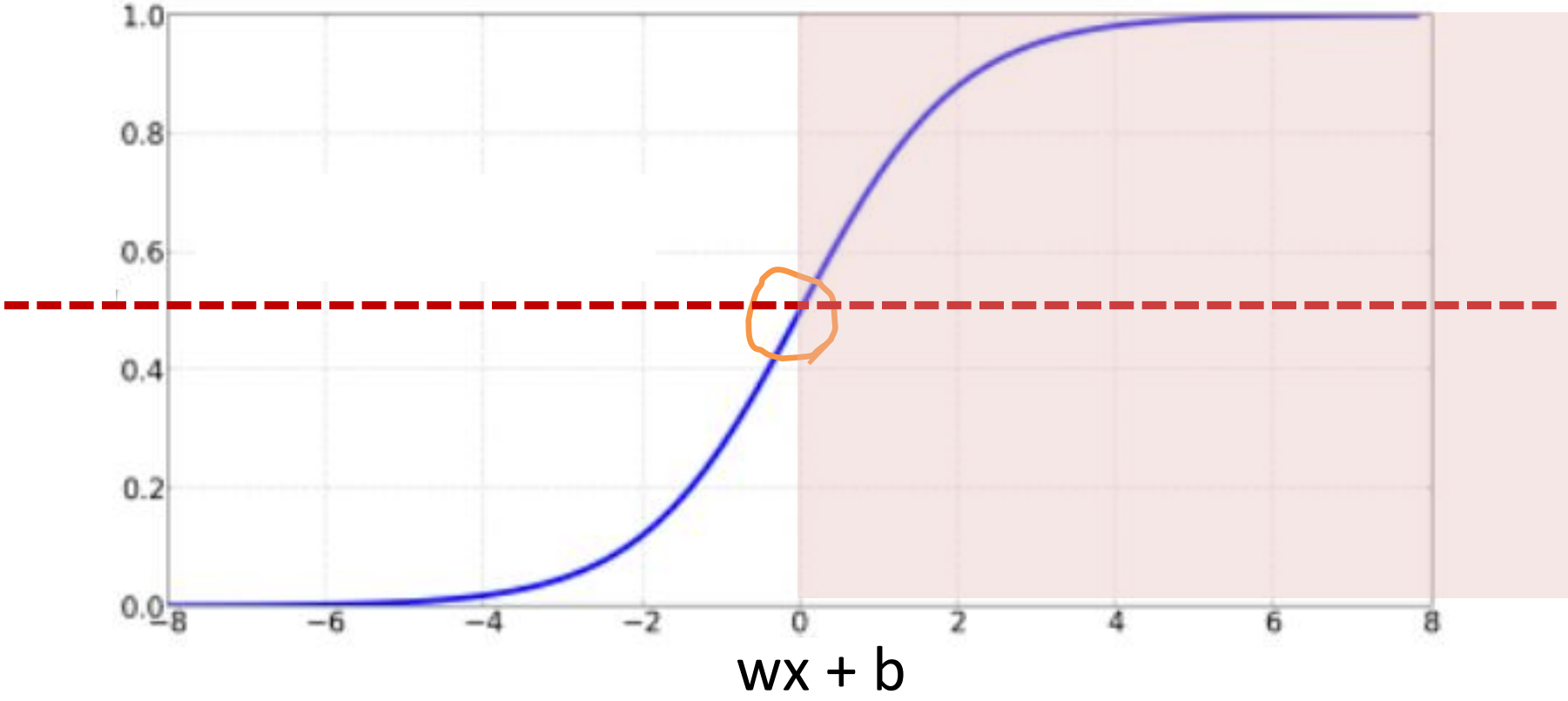
$$z = w \cdot x + b$$



Sigmoid函数

$$P(y=1) = \sigma(w \cdot x + b) = \frac{1}{1+e^{-(w \cdot x+b)}}$$

$P(y=1)$

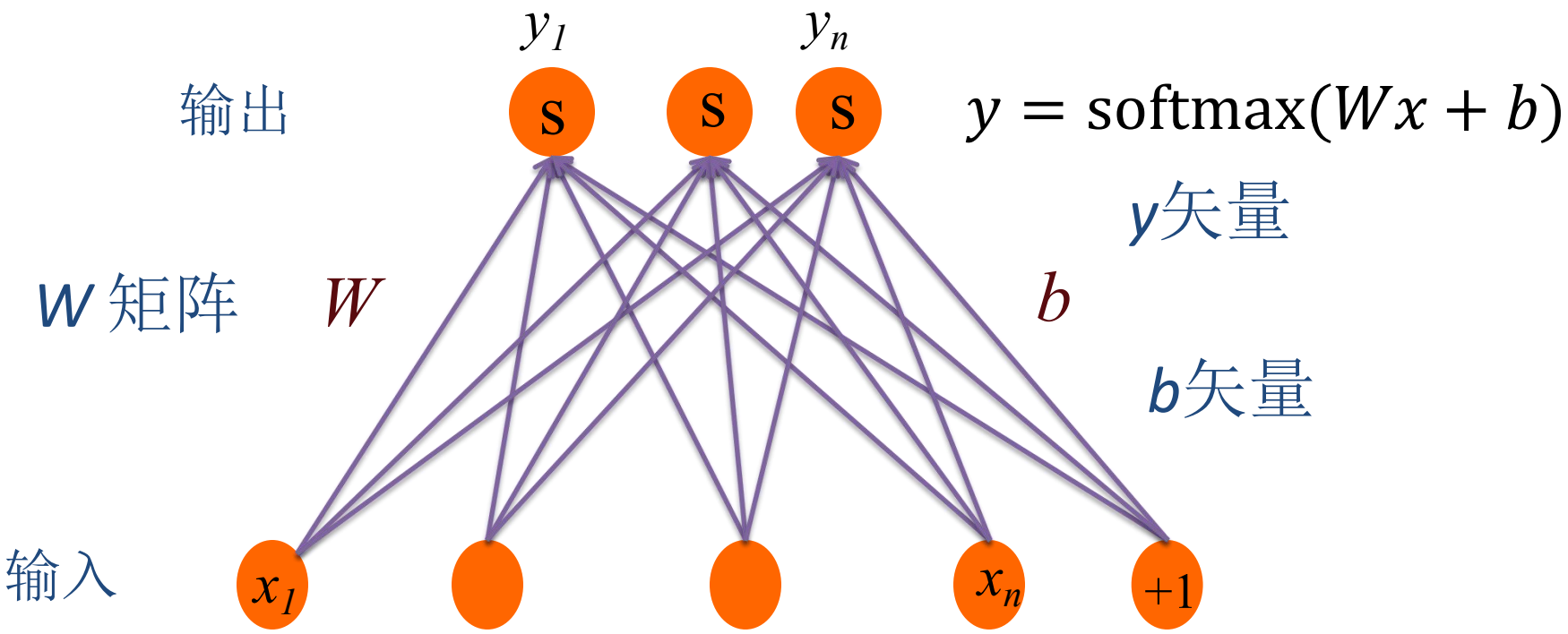


Logistics回归：决策

$$\hat{y} = \begin{cases} 1 & \text{if } P(y=1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

这里0.5称为决策边界

多元Logistics回归



多元Logistics回归：**softmax**

将k维响应向量$z = [z_1, z_2, \ldots, z_k]$归一化为概率分布

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^{k} \exp(z_j)} \quad 1 \le i \le k$$

$$\text{softmax}(z) = \left[\frac{\exp(z_1)}{\sum_{i=1}^{k} \exp(z_i)}, \frac{\exp(z_2)}{\sum_{i=1}^{k} \exp(z_i)}, \ldots, \frac{\exp(z_k)}{\sum_{i=1}^{k} \exp(z_i)} \right]$$

多元Logistics回归

– 将向量 $z = [z_1, z_2, ..., z_k]$ 转化为概率向量

$$z = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$$

$$\text{softmax}(z) = \left[\frac{\exp(z_1)}{\sum_{i=1}^{k} \exp(z_i)}, \frac{\exp(z_2)}{\sum_{i=1}^{k} \exp(z_i)}, \ldots, \frac{\exp(z_k)}{\sum_{i=1}^{k} \exp(z_i)} \right]$$

$$[0.055, 0.090, 0.0067, 0.10, 0.74, 0.010]$$

多元Logistics回归

$$p(y=c|x) = \frac{\exp(w_c \cdot x + b_c)}{\sum_{j=1}^{k} \exp(w_j \cdot x + b_j)}$$

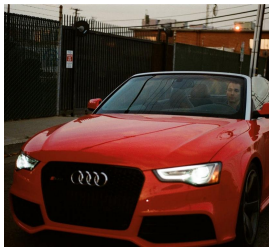
输入仍然是权重向量*w*和输入向量*x*的乘积
输出是属于各个类别的概率.

本章内容

- 5.1.为什么需要学习
- 5.2.最近邻方法
- 5.3.线性分类器
- 5.4.损失函数与正则化
- 5.5.最优化

目前为止：定义一个线性响应函数

$$f(x,W) = Wx + b$$

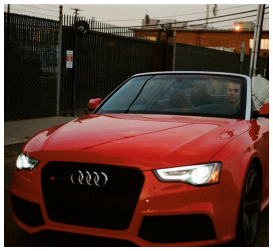

airplane	-3.45	-0.51	3.42
automobile	-8.87	**6.04**	4.64
bird	0.09	5.31	2.65
cat	**2.9**	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	**-4.34**
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

给定 W, 我们可以计算图像x的类别响应分值.

但是如何选择最优的 W?

选择最优的 W

$$f(x,W) = Wx + b$$


airplane	-3.45	-0.51	3.42
automobile	-8.87	**6.04**	4.64
bird	0.09	5.31	2.65
cat	**2.9**	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	**-4.34**
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

方法:

1. 定义一个衡量W好坏的损失函数

2. 找出使得损失函数最小的W (**最优化**)

损失函数

损失函数告诉我们当前的分类器有多好

低损失= 好分类器
高损失= 坏分类器

(同样称为: 目标函数; 代价函数)

损失函数

损失函数告诉我们当前的分类器有多好

低损失= 好分类器
高损失= 坏分类器

(同样称为: 目标函数; 代价函数)

负向损失函数也被称为**报酬函数**, **利润函数**, **效用函数**, **拟合度函数**, 等等

损失函数

损失函数告诉我们当前的分类器有多好

低损失= 好分类器
高损失= 坏分类器

(同样称为: 目标函数; 代价函数)

负向损失函数也被称为**报酬函数**, **利润函数**, **效用函数**, **拟合度函数**, 等等

给定数据集

$$\{(x_i, y_i)\}_{i=1}^{N}$$

这里 x_i 对应图像
y_i 对应标签

损失函数

损失函数告诉我们当前的分类器有多好

低损失= 好分类器
高损失= 坏分类器

(同样称为: 目标函数; 代价函数)

负向损失函数也被称为**报酬函数, 利润函数, 效用函数, 拟合度函数**, 等等

给定数据集

$$\{(x_i, y_i)\}_{i=1}^{N}$$

这里 x_i 对应图像
y_i 对应标签

一个样例的损失对应为

$$L_i(f(x_i, W), y_i)$$

损失函数

损失函数告诉我们当前的分类器有多好

低损失= 好分类器
高损失= 坏分类器

(同样称为: 目标函数; 代价函数)

负向损失函数也被称为**报酬函数, 利润函数, 效用函数, 拟合度函数**, 等等

给定数据集

$$\{(x_i, y_i)\}_{i=1}^{N}$$

这里 x_i 对应图像
y_i 对应标签

一个样例的损失对应为

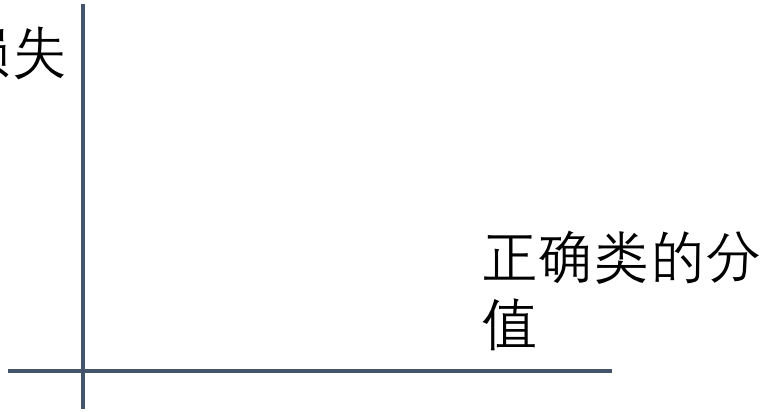
$$L_i(f(x_i, W), y_i)$$

最终损失为各样例损失的平均值:

$$L = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

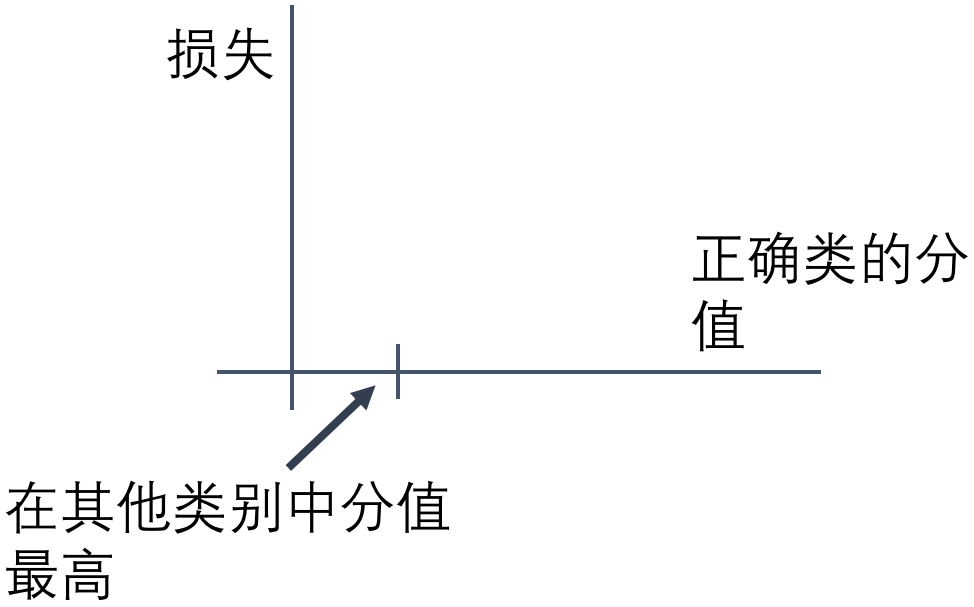
多类 SVM 损失

”当前正确类的分值应该高于任何其他类的分值”



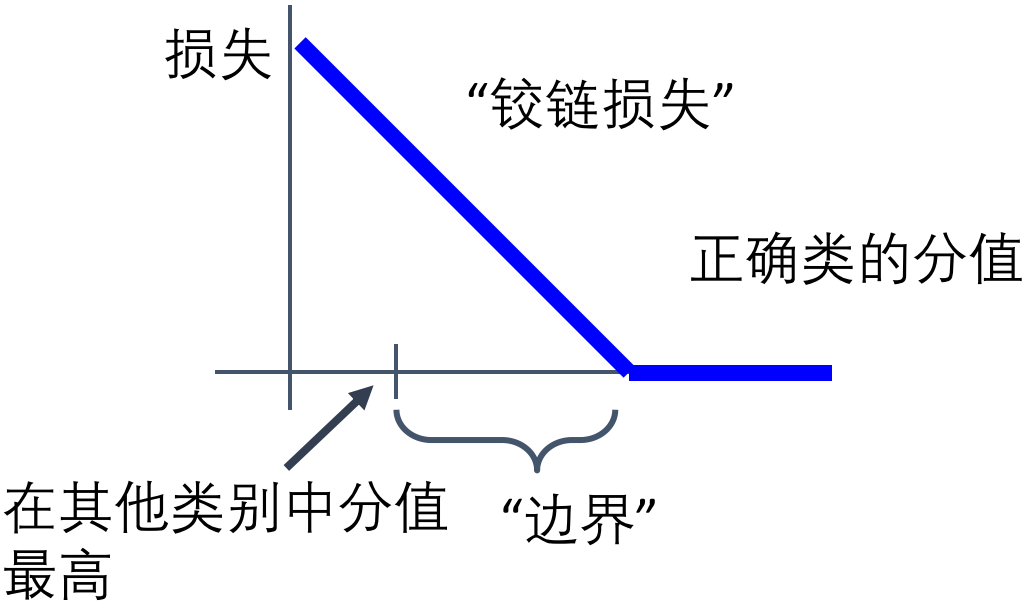
多类 SVM 损失

”当前正确类的分值应该高于任何其他类的分值”



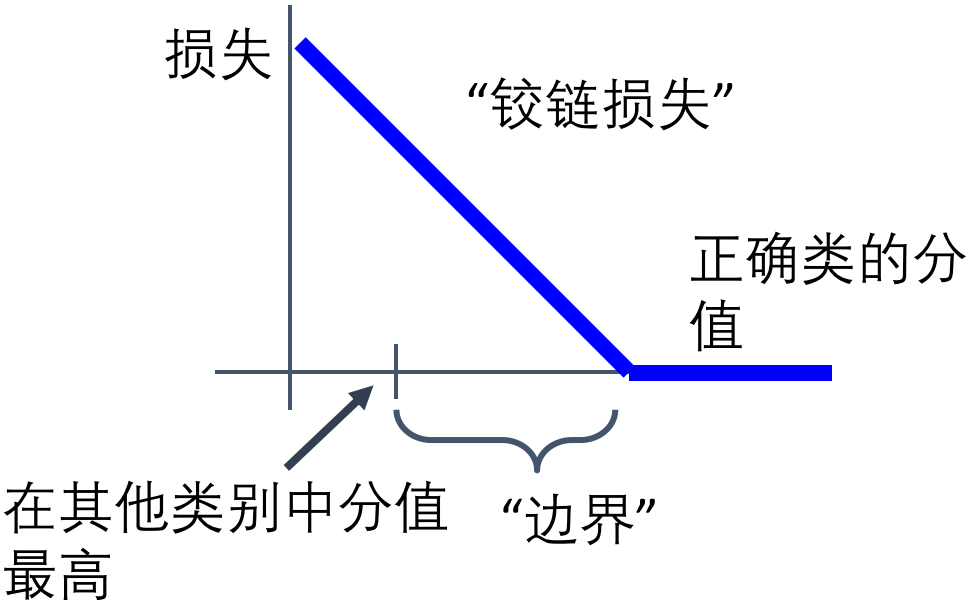
多类 SVM 损失

”当前正确类的分值应该高于任何其他类的分值”



多类 SVM 损失

”当前正确类的分值应该高于任何其他类的分值”



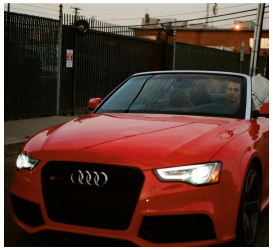
给定一副图像 (x_i, y_i)
(x_i 为图像, y_i 为标签)

令 $s = f(x_i, W)$ 为响应分值

则SVM 损失定义为:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

多类 SVM 损失


cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

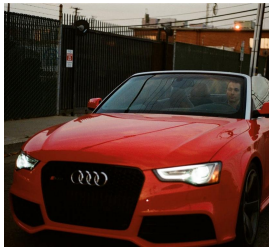
给定一副图像 (x_i, y_i)
(x_i 为图像, y_i 为标签)

令 $s = f(x_i, W)$ 为响应分值

则SVM 损失定义为:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

多类 SVM 损失


cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
Loss	2.9		

给定一副图像 (x_i, y_i)
(x_i 为图像, y_i 为标签)

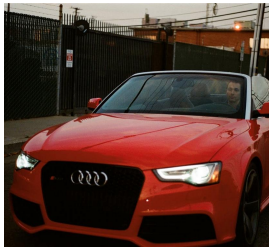
令 $s = f(x_i, W)$为响应分值

则SVM 损失定义为:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

= max(0, 5.1 - 3.2 + 1)
+ max(0, -1.7 - 3.2 + 1)
= max(0, 2.9) + max(0, -3.9)
= 2.9 + 0
= 2.9

多类 SVM 损失


cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
Loss	2.9	0	

给定一副图像 (x_i, y_i)
(x_i 为图像, y_i 为标签)

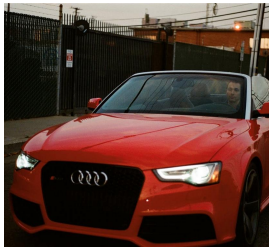
令 $s = f(x_i, W)$为响应分值

则SVM 损失定义为:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

= max(0, 1.3 - 4.9 + 1)
+max(0, 2.0 - 4.9 + 1)
= max(0, -2.6) + max(0, -1.9)
= 0 + 0
= 0

多类 SVM 损失


cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
Loss	2.9	0	12.9

给定一副图像 (x_i, y_i)
(x_i 为图像, y_i 为标签)

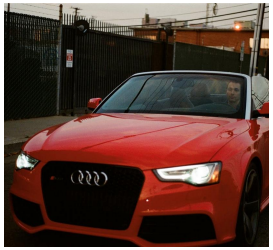
令 $s = f(x_i, W)$ 为响应分值

则SVM 损失定义为:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

= max(0, 2.2 - (-3.1) + 1)
+max(0, 2.5 - (-3.1) + 1)
= max(0, 6.3) + max(0, 6.6)
= 6.3 + 6.6
= 12.9

多类 SVM 损失


cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
Loss	2.9	0	12.9

给定一副图像 (x_i, y_i)
(x_i 为图像, y_i 为标签)

令 $s = f(x_i, W)$ 为响应分值

则SVM 损失定义为:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

整个数据集的损失为:

L = (2.9 + 0.0 + 12.9) / 3
= 5.27

交叉熵损失(多元Logistic回归)

将分类器输出的分值解释为概率:

cat	**3.2**
car	5.1
frog	-1.7

交叉熵损失(多元Logistic回归)

将分类器输出的分值解释为概率:

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax 函数

cat	**3.2**
car	5.1
frog	-1.7

交叉熵损失(多元Logistic回归)

将分类器输出的分值解释为概率:

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax 函数

cat	**3.2**
car	5.1
frog	-1.7

非归一化 log-概率 / logits

交叉熵损失(多元Logistic回归)

将分类器输出的分值解释为概率:

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax 函数

概率必须 >= 0

cat	**3.2**	exp	**24.5**
car	5.1		164.0
frog	-1.7		0.18

非归一化 log-概率 / logits

非归一化概率

交叉熵损失(多元Logistic回归)

将分类器输出的分值解释为概率:

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax 函数

概率必须 >= 0　　概率和为1

	非归一化 log-概率 / logits	exp	非归一化概率	归一化	概率
cat	**3.2**	→	**24.5**	→	**0.13**
car	5.1		164.0		0.87
frog	-1.7		0.18		0.00

交叉熵损失(多元Logistic回归)

将分类器输出的分值解释为概率:

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax 函数

概率必须 >= 0

概率和为1

$$L_i = -\log P(Y = y_i|X = x_i)$$

	非归一化 log-概率 / logits	exp	非归一化概率	归一化	概率
cat	**3.2**	→	**24.5**	→	**0.13**
car	5.1	→	164.0	→	0.87
frog	-1.7	→	0.18	→	0.00

L_i = -log(0.13)
= **2.04**

交叉熵损失(多元Logistic回归)

将分类器输出的分值解释为概率:

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax 函数

概率必须 >= 0

概率和为1

$$L_i = -\log P(Y = y_i|X = x_i)$$

	非归一化 log-概率 / logits	exp	非归一化概率	归一化	概率
cat	**3.2**	→	**24.5**	→	**0.13**
car	5.1	→	164.0	→	0.87
frog	-1.7	→	0.18	→	0.00

L_i = -log(0.13)
= **2.04**

最大似然估计
选择权重对应观察数据的最大似然函数

交叉熵损失(多元Logistic回归)

将分类器输出的分值解释为概率:

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax 函数

概率必须 >= 0

概率和为1

$$L_i = -\log P(Y = y_i|X = x_i)$$

	非归一化 log-概率 / logits	exp	非归一化概率	归一化	概率	比较	正确概率
cat	**3.2**	→	**24.5**	→	**0.13**	→	**1.00**
car	5.1	→	164.0	→	0.87	→	0.00
frog	-1.7	→	0.18	→	0.00	→	0.00

交叉熵损失(多元Logistic回归)

将分类器输出的分值解释为概率:

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax 函数

概率必须 >= 0

概率和为1

$$L_i = -\log P(Y = y_i|X = x_i)$$

	非归一化 log-概率 / logits	exp	非归一化概率	归一化	概率	比较	正确概率
cat	3.2		24.5		0.13		1.00
car	5.1		164.0		0.87		0.00
frog	-1.7		0.18		0.00		0.00

Kullback–Leibler 散度

$$D_{KL}(P||Q) = \sum_y P(y) \log \frac{P(y)}{Q(y)}$$

交叉熵损失(多元Logistic回归)

将分类器输出的分值解释为概率:

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax 函数

概率必须 >= 0

概率和为1

$$L_i = -\log P(Y = y_i|X = x_i)$$

	非归一化 log-概率 / logits	exp	非归一化概率	归一化	概率	比较	正确概率
cat	**3.2**	→	**24.5**	→	**0.13**	←	**1.00**
car	5.1	→	164.0	→	0.87		0.00
frog	-1.7	→	0.18	→	0.00		0.00

交叉熵

$$H(P, Q) = H(p) + D_{KL}(P||Q)$$

交叉熵损失(多元Logistic回归)

将分类器输出的分值解释为概率:

cat **3.2**

car 5.1

frog -1.7

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax 函数

最大化正确类别的概率

$$L_i = -\log P(Y = y_i|X = x_i)$$

把所有的都结合起来:

$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$

交叉熵损失(多元Logistic回归)

将分类器输出的分值解释为概率:

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax函数

cat	**3.2**
car	5.1
frog	-1.7

最大化正确类别的概率

$$L_i = -\log P(Y = y_i|X = x_i)$$

把所有的都结合起来:

$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$

Q: 损失L_i的最小和最大值是?

交叉熵损失(多元Logistic回归)

将分类器输出的分值解释为概率:

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax 函数

cat	**3.2**
car	5.1
frog	-1.7

最大化正确类别的概率

$$L_i = -\log P(Y = y_i|X = x_i)$$

把所有的都结合起来:

$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$

Q:损失L_i的最小和最大值是?

A: 最小 0, 最大 +∞

交叉熵损失(多元Logistic回归)

将分类器输出的分值解释为概率:

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax函数

cat	**3.2**
car	5.1
frog	-1.7

最大化正确类别的概率

$$L_i = -\log P(Y = y_i|X = x_i)$$

把所有的都结合起来:

$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$

Q: 如果所有分值是小的随机数，损失为多少?

交叉熵损失(多元Logistic回归)

将分类器输出的分值解释为概率:

$$s = f(x_i; W)$$

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax函数

cat	**3.2**
car	5.1
frog	-1.7

最大化正确类别的概率

$$L_i = -\log P(Y = y_i|X = x_i)$$

把所有的都结合起来:

$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$

Q: 如果所有分值是小的随机数，损失为多少?

A: -log(1/C)
log(10) ≈ 2.3

交叉熵 vs SVM 损失

$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

假设响应分值:
[10, -2, 3]
[10, 9, 9]
[10, -100, -100]
并且 $y_i = 0$

Q: 交叉熵损失为多少?
SVM损失?

交叉熵 vs SVM 损失

$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

假设响应分值:
[10, -2, 3]
[10, 9, 9]
[10, -100, -100]
并且 $y_i = 0$

Q:交叉熵损失为多少?
SVM损失?

A: Cross-entropy 损失 > 0
SVM 损失 = 0

交叉熵 vs SVM 损失

$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

假设响应分值:
[10, -2, 3]
[10, 9, 9]
[10, -100, -100]
并且 $y_i = 0$

Q: 如果稍微改变最后一个数据的值，损失有什么变化?

交叉熵 vs SVM 损失

$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

假设响应分值:
[10, -2, 3]
[10, 9, 9]
[10, -100, -100]
并且 $y_i = 0$

Q:如果稍微改变最后一个数据的值，损失有什么变化?

A: 交叉熵损失将改变;
SVM损失将保持不变

交叉熵 vs SVM 损失

$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

假设响应分值:

[10, -2, 3]

[10, 9, 9]

[10, -100, -100]

并且 $y_i = 0$

Q:如果正确类别的响应值从10变化到20，损失有什么变化?

交叉熵 vs SVM 损失

$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

假设响应分值:
[10, -2, 3]
[10, 9, 9]
[10, -100, -100]
并且 $y_i = 0$

Q:如果正确类别的响应值从10变化到20，损失有什么变化?

A: 交叉熵损失将降低,
SVM 损失仍为0

正则化: 训练错误之外的损失

$$L(W) = \underbrace{\frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, W), y_i)}$$

数据损失: 模型预测结果应该匹配训练数据

正则化: 训练错误之外的损失

$$L(W) = \underbrace{\frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, W), y_i)}_{\text{数据损失}} + \underbrace{\lambda R(W)}_{\text{正则化}}$$

数据损失: 模型预测结果应该匹配训练数据

正则化: 阻止模型在训练数据上过拟合

正则化: 训练错误之外的损失

$$L(W) = \underbrace{\frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, W), y_i)}_{\text{数据损失}} + \underbrace{\lambda R(W)}_{\text{正则化}}$$

λ = 正则化因子 (超参数)

数据损失: 模型预测结果应该匹配训练数据

正则化:阻止模型在训练数据上过拟合

正则化: 训练错误之外的损失

$$L(W) = \underbrace{\frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, W), y_i)}_{\text{数据损失}} + \underbrace{\lambda R(W)}_{\text{正则化}}$$

λ = 正则化因子 (超参数)

数据损失: 模型预测结果应该匹配训练数据

正则化:阻止模型在训练数据上过拟合

简单示例

L2 正则化: $R(W) = \sum_k \sum_l W_{k,l}^2$

L1正则化: $R(W) = \sum_k \sum_l |W_{k,l}|$

结合(L1 + L2): $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

更复杂:

Dropout

Batch归一化

正则化: 训练错误之外的损失

$$L(W) = \underbrace{\frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, W), y_i)}_{\text{数据损失}} + \underbrace{\lambda R(W)}_{\text{正则化}}$$

λ = 正则化因子 (超参数)

数据损失: 模型预测结果应该匹配训练数据

正则化:阻止模型在训练数据上过拟合

正则项目的:

- 表达模型选择的倾向性除了”最小化训练错误”
- 避免过拟合: 倾向简单泛化性好的模型
- 通过增加变化提高优化效率

正则化: 训练错误之外的损失

$$x = [1, 1, 1, 1]$$

$$w_1 = [1, 0, 0, 0]$$

$$w_2 = [0.25, 0.25, 0.25, 0.25]$$

$$w_1^T x = w_2^T x = 1$$

L2正则化

$$R(W) = \sum_k \sum_l W_{k,l}^2$$

正则化: 训练错误之外的损失

$$x = [1, 1, 1, 1]$$

$$w_1 = [1, 0, 0, 0]$$

$$w_2 = [0.25, 0.25, 0.25, 0.25]$$

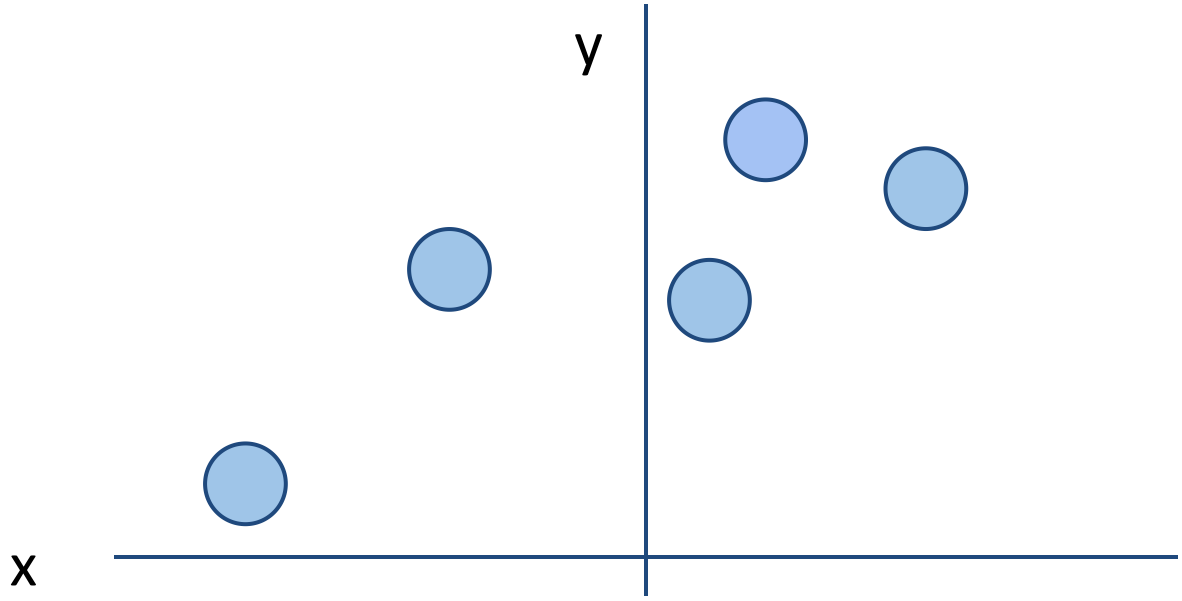
$$w_1^T x = w_2^T x = 1$$

L2正则化

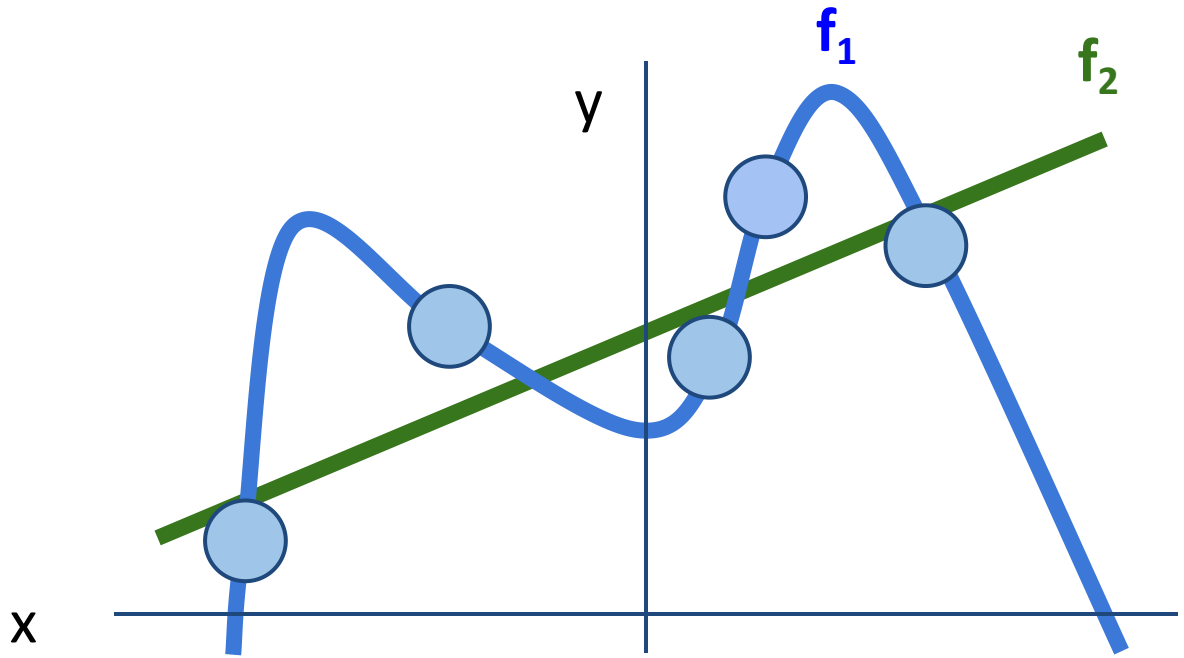
$$R(W) = \sum_k \sum_l W_{k,l}^2$$

L2 倾向于均匀分布的系数

正则项： 数据拟合

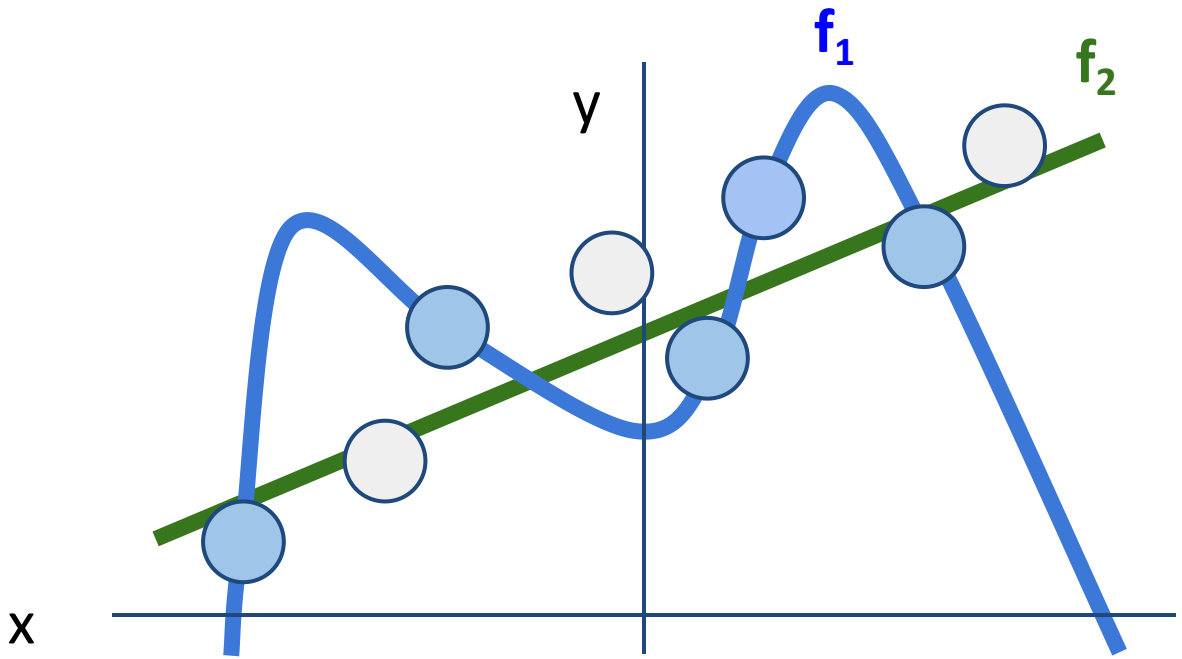


正则项： 数据拟合



模型 f_1 完美的拟合数据
模型 f_2 有偏差，但是更简单

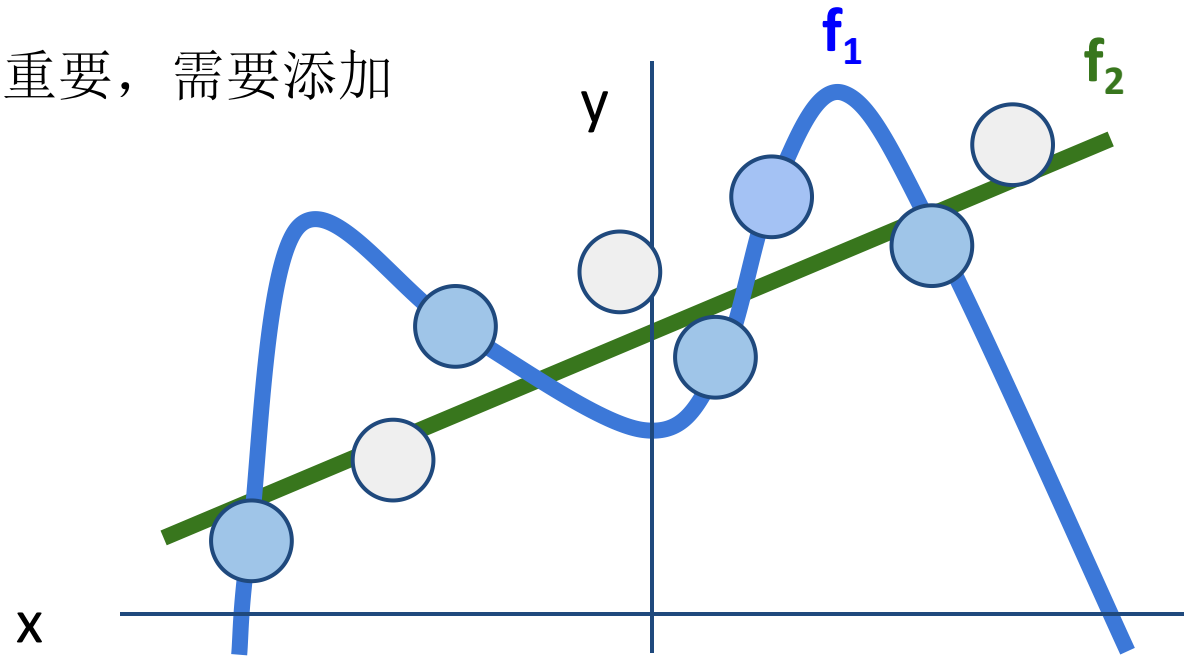
正则项： 数据拟合



正则项增加模型参数相关的惩罚项

正则项： 数据拟合

正则项很重要，需要添加



正则项增加模型参数相关的惩罚项，避免过拟合

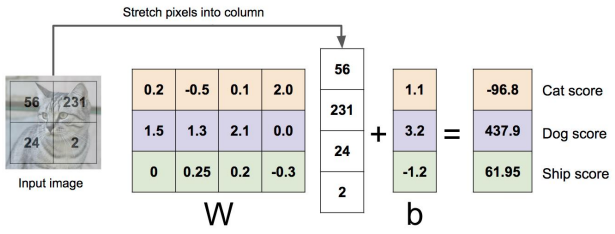
本章内容

- 5.1.为什么需要学习
- 5.2.最近邻方法
- 5.3.线性分类器
- 5.4.损失函数与正则化
- 5.5.最优化

回顾：线性分类器的三个角度

代数角度

$f(x,W) = Wx$

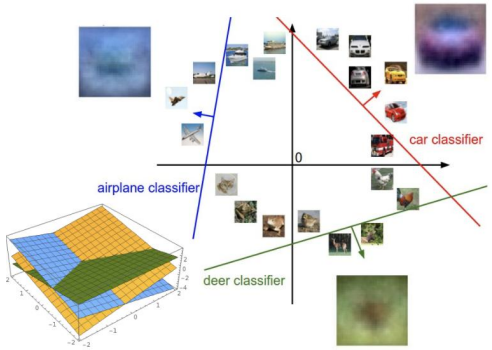


可视化角度

每类一个模板

几何角度

超平面切割



回顾: 损失函数

- 数据(x, y)
- 一个响应分值函数:
- 损失函数:

$$s = f(x; W) = Wx$$

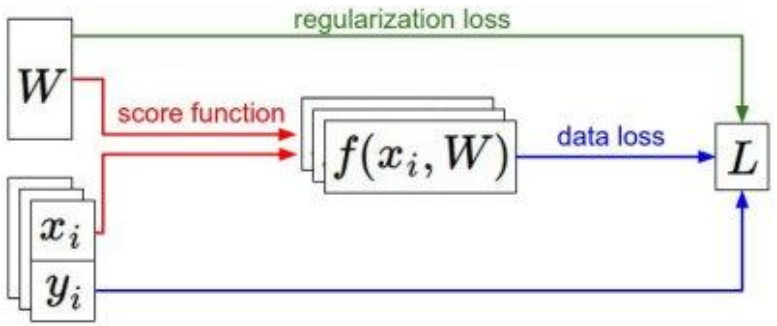
线性分类器

$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$ Softmax

SVM

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$L = \frac{1}{N}\sum_{i=1}^{N} L_i + R(W)$$ 全损失



回顾: 损失函数

- 数据(x, y)
- 一个响应分值函数:
- 损失函数:

Q: 如何确定最优的 W?

$$s = f(x; W) = Wx$$

线性分类器

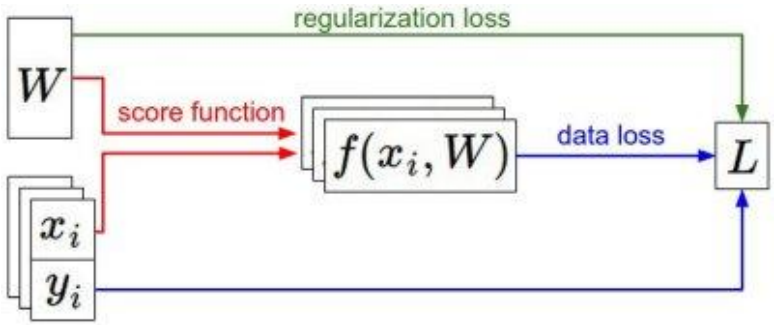
A: 如下

$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$ Softmax

SVM

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$L = \frac{1}{N}\sum_{i=1}^{N} L_i + R(W)$$ 全损失



最优化

- 寻找一个最优的W，使得损失最小

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, W), y_i) + \lambda R(W)$$

损失包括数据偏差和正则项

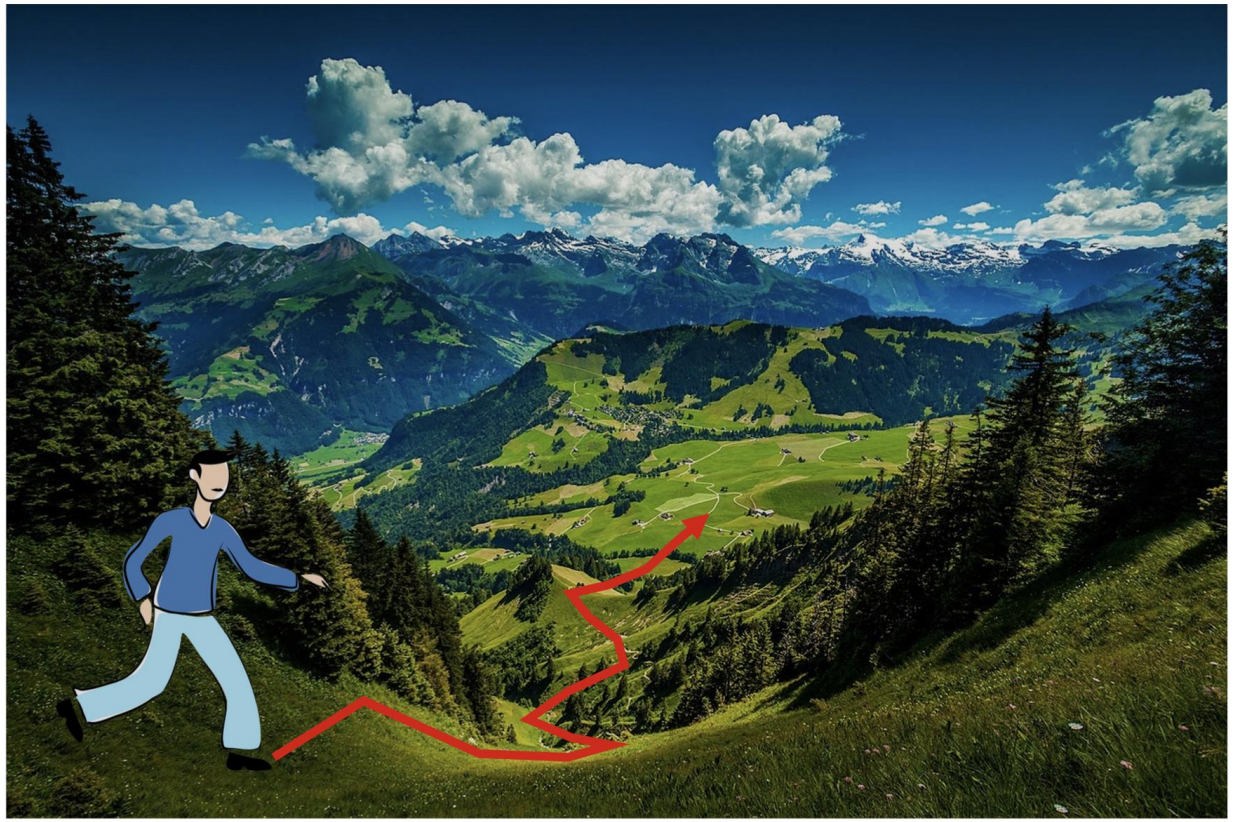
最优化

- 模型

Optimization

$$w^* = \arg\min_w L(w)$$

最优化



沿着梯度方向下降

在 1-维情况下, 导数为坡度方向:

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

在多维情况下, 坡度为梯度方向（多个维度偏导数的组合）

任意方向的变化率为梯度和方向向量的内积
最速下降方向为负梯度方向

损失为关于W的函数: **分析梯度**

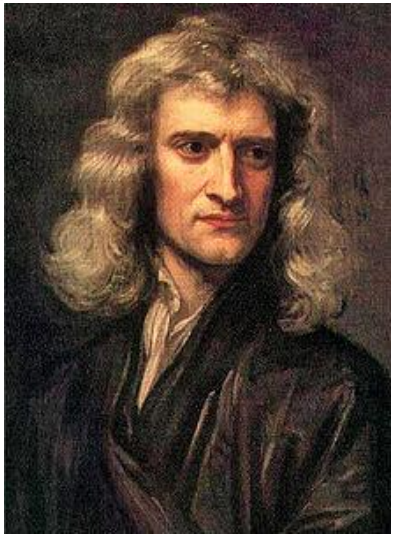
$$L = \frac{1}{N}\sum_{i=1}^{N} L_i + \sum_k W_k^2$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$s = f(x; W) = Wx$$

想求 $\nabla_W L$

计算梯度



This image is in the public domain

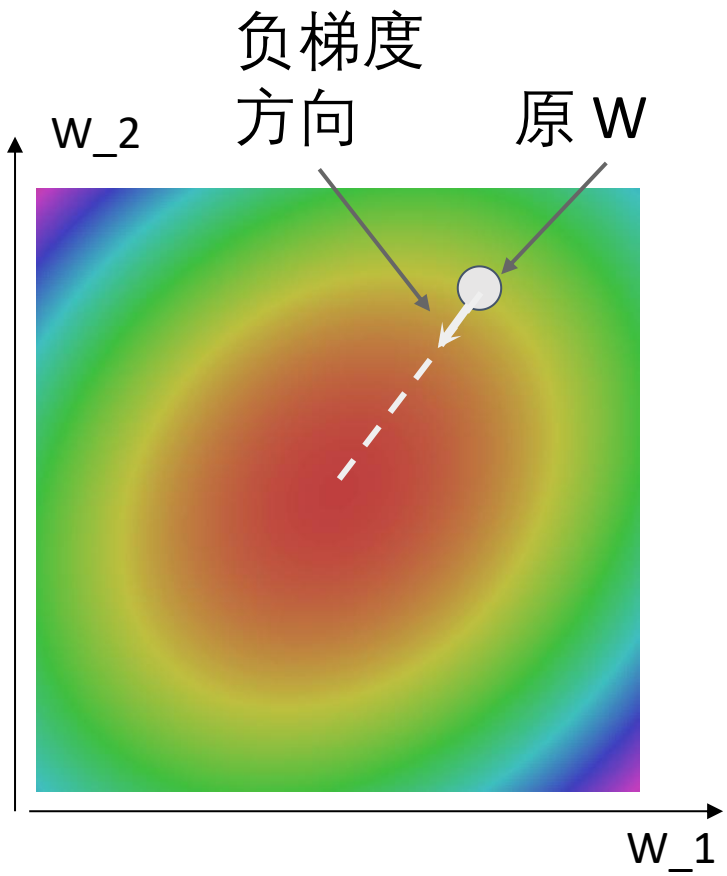
This image is in the public domain

梯度速降

沿着梯度方向进行迭代(局部下降最快方向)

```
# Vanilla gradient descent
w = initialize_weights()
for t in range(num_steps):
  dw = compute_gradient(loss_fn, data, w)
  w -= learning_rate * dw
```

超参数:
- 权重初始化方法
- 步数(迭代次数)
- 学习速率

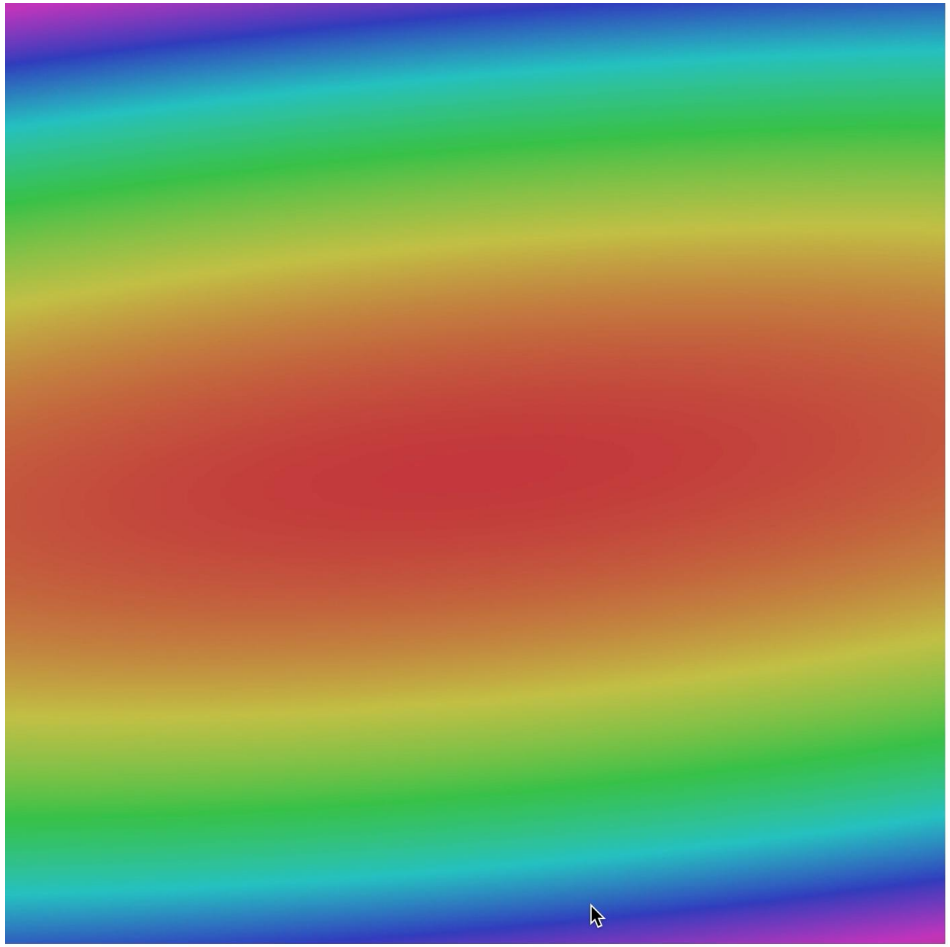


梯度速降

沿着梯度方向进行迭代(局部下降最快方向)

```
# Vanilla gradient descent
w = initialize_weights()
for t in range(num_steps):
  dw = compute_gradient(loss_fn, data, w)
  w -= learning_rate * dw
```

超参数:
- 权重初始化方法
- 步数(迭代次数)
- 学习速率



批梯度速降

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(x_i, y_i, W) + \lambda R(W)$$

$$\nabla_W L(W) = \frac{1}{N}\sum_{i=1}^{N} \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

当 N 很大时，全部求和代价高!

随机梯度速降(SGD)

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(x_i, y_i, W) + \lambda R(W)$$

$$\nabla_W L(W) = \frac{1}{N}\sum_{i=1}^{N} \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

```
# Stochastic gradient descent
w = initialize_weights()
for t in range(num_steps):
  minibatch = sample_data(data, batch_size)
  dw = compute_gradient(loss_fn, minibatch, w)
  w -= learning_rate * dw
```

当 N 很大时，全部求和代价高!

使用一个很小的批进行求和来近似
通常 32 / 64 / 128

超参数:
- 权重初始化
- 步数
- 学习率
- 批大小
- 数据采样

随机梯度速降(SGD)

$$L(W) = \mathbb{E}_{(x,y)\sim p_{data}}[L(x, y, W)] + \lambda R(W)$$

可以看成全局损失的期望值
数据分布 p_{data}

$$\approx \frac{1}{N}\sum_{i=1}^{N} L(x_i, y_i, W) + \lambda R(W)$$

通过采样来近似期望

随机梯度速降(SGD)

$$L(W) = \mathbb{E}_{(x,y)\sim p_{data}}\left[L(x, y, W)\right] + \lambda R(W)$$

可以看成全局损失的期望值
数据分布 p_{data}

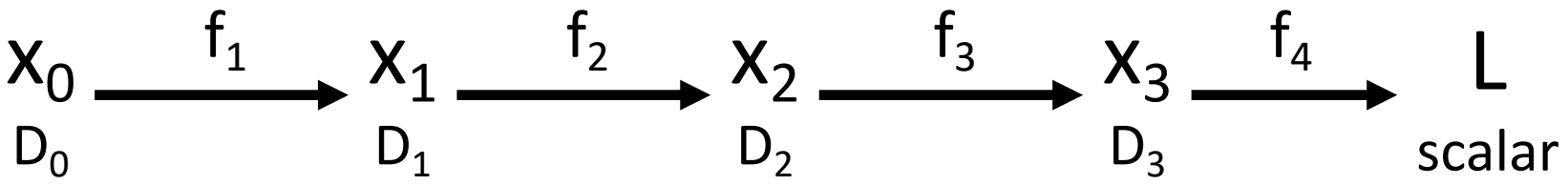
$$\approx \frac{1}{N}\sum_{i=1}^{N} L(x_i, y_i, W) + \lambda R(W)$$

通过采样来近似期望

$$\nabla_W L(W) = \nabla_W \mathbb{E}_{(x,y)\sim p_{data}}\left[L(x, y, W)\right] + \lambda \nabla_W R(W))$$

$$\approx \sum_{i=1}^{N} \nabla_W L_W(x_i, y_i, W) + \nabla_W R(W)$$

回顾: 求导链式法则



链式法则

$$\frac{\partial L}{\partial x_0} = \left(\frac{\partial x_1}{\partial x_0}\right)\left(\frac{\partial x_2}{\partial x_1}\right)\left(\frac{\partial x_3}{\partial x_2}\right)\left(\frac{\partial L}{\partial x_3}\right)$$

$D_0 \times D_1$ $D_1 \times D_2$ $D_2 \times D_3$ D_3

计算 标量输出的梯度
w/为所有矢量输入

本章小结

- 为什么需要学习
  - 图像分类的挑战
  - 传统图像分类问题
  - 机器学习方法
- 最近邻方法
  - 最近邻分类方法
  - 距离测度
  - 超参数
- 贝叶斯分类方法
  - 贝叶斯分类器模型
  - 朴素贝叶斯
  - 朴素多元贝叶斯
- 线性分类与Logistic回归
  - 线性分类器
  - 线性分类器的三种解释
  - Logistic回归
- 损失函数与正则化
  - SVM损失函数
  - 交叉熵损失函数
  - 正则化
- 最优化
  - 梯度速降法
  - 随机梯度速降法

结束

下一次课

- 神经网络